

ОПТИМИЗАЦИОННЫЕ АЛГОРИТМЫ НА ГРАФАХ

Цель: освоение навыков решения задач с применением оптимизационных алгоритмов на графах.

Задачи:

- изучение теоретических основ оптимизационных алгоритмов на графах;
- освоить практическое применение оптимизационных алгоритмов на графах.

План лекции

1. Алгоритм поиска в ширину;
2. Алгоритм поиска в глубину;
3. Топологическая сортировка.



ОПТИМИЗАЦИОННЫЕ АЛГОРИТМЫ НА ГРАФАХ

Многие алгоритмы на графах могут быть решены путем систематического перебора (обхода) вершин и дуг графа. Такой обход можно выполнить многими способами. На практике широкое распространение получили два способа, получивших специальные названия: **поиск в ширину** и **поиск в глубину**.

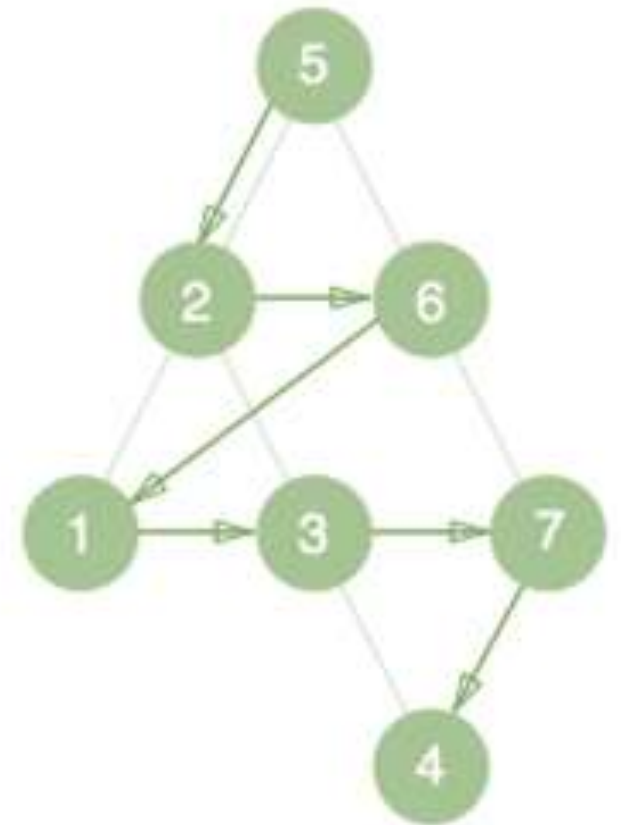
*Для обозначения алгоритма поиска в ширину обычно используют аббревиатуру **BFS (Breadth-first search)**.*

*Для обозначения алгоритма поиска в глубину используют аббревиатуру **DFS (Depth-first search)**.*

Алгоритм поиска в ширину (BFS, breadth-first search)

Алгоритм подразумевает, что задана исходная (**стартовой**) вершина, и основывается на простом правиле: при выборе очередной вершины предпочтение отдается ближайшей. При этом считается, что все дуги графа имеют единичную длину.

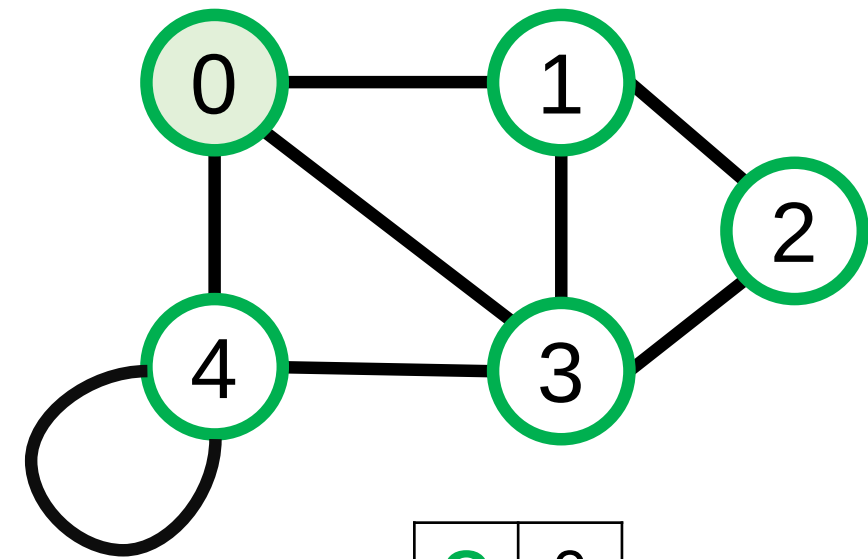
Сначала посещается стартовая вершина, затем все вершины, смежные ей (т. е. находящиеся на расстоянии 1), после чего вершины, находящиеся на расстоянии 2 от стартовой и т.д.



Breadth-first traversal

Алгоритм поиска в ширину (BFS, breadth-first search)

Очевидно, что алгоритм **BFS** посетит только те вершины, для которых существует последовательность дуг, связывающая с ними стартовую вершину. Обычно, по окончании работы алгоритма, осуществляется проверка на полноту обхода и, если имеются вершины, которые не посещались, то среди них выбирается любая, назначается стартовой и алгоритм выполняется снова.



Q	0
----------	---

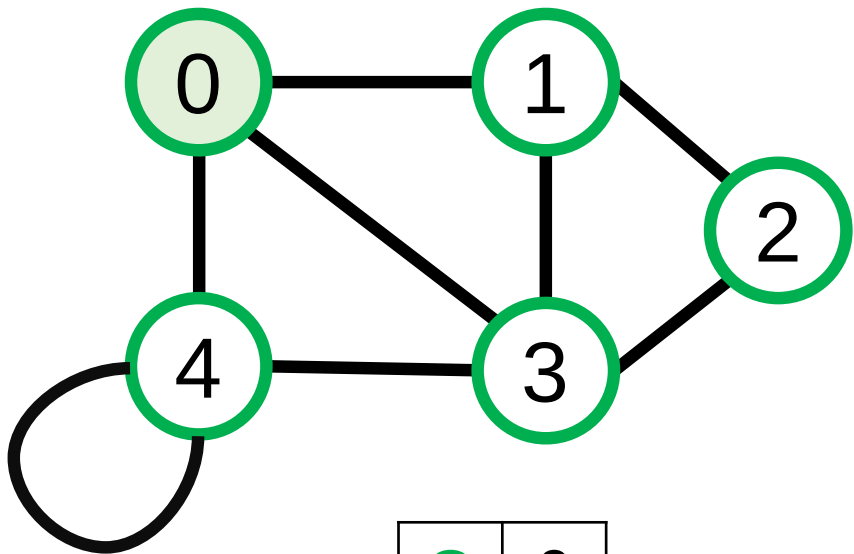
C	G	W	W	W	W
----------	---	---	---	---	---

D	0	I	I	I	I
----------	---	---	---	---	---

P	N	N	N	N	N
----------	---	---	---	---	---

Текущее состояние алгоритма хранится в следующих структурах памяти: **Q** – очередь вершин, **C** – массив окраски вершин, **D** – массив расстояний и **P** – массив предшествующих вершин.

Очередь **Q** (структура памяти, реализующая алгоритм «первый вошел – первый вышел»), используется для промежуточного хранения номеров вершин. На каждом шаге алгоритма, в очередь помещаются номера вершин в порядке их обнаружения. На каждом шаге, кроме первого, из очереди извлекается очередной номер вершины, подлежащей отметке о посещении. На первом шаге алгоритма в очередь помещается номер стартовой вершины. На последнем шаге очередь пуста.



Q	0
---	---

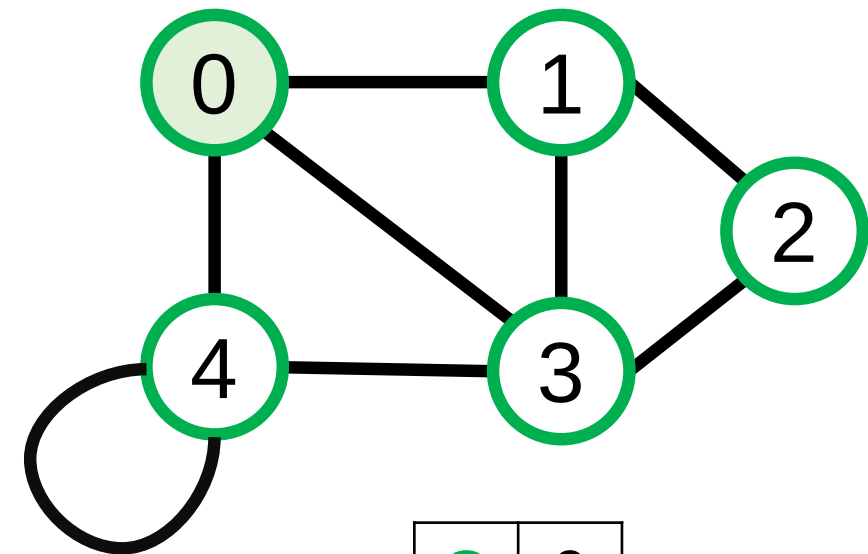
C	G	W	W	W	W
---	---	---	---	---	---

D	0	I	I	I	I
---	---	---	---	---	---

P	N	N	N	N	N
---	---	---	---	---	---

Массив **C** используется для хранения состояния вершин. С каждым из трех возможных состояний обычно связывают цвет: белый (**W**) – вершина не посещалась, серый (**G**) – вершина посещалась, черный (**B**) – фиксирован факт посещения вершины. На первом шаге алгоритма стартовая вершина окрашивается в серый цвет, а остальные – в белый. На последнем шаге все вершины становятся черными.

В массиве **D** для каждой вершины хранятся расстояния от стартовой вершины. На первом шаге для стартовой вершины в массиве **D** устанавливается значение 0, а для остальных вершин – значение «бесконечность» (**I**). На последнем шаге алгоритма для всех доступных вершин будут заполнены значения, равные их расстоянию от стартовой вершины.



Q	0
----------	---

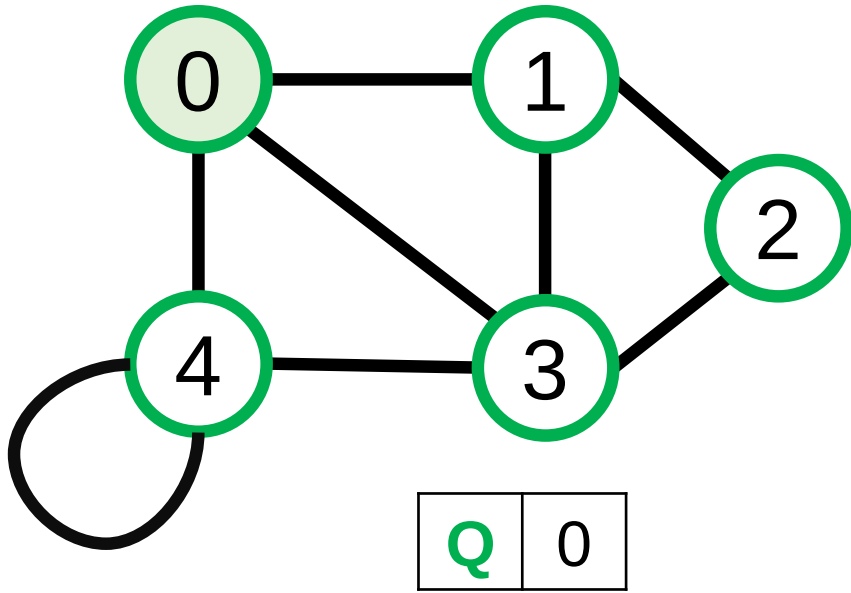
C	G	W	W	W	W
----------	---	---	---	---	---

D	0	I	I	I	I
----------	---	---	---	---	---

P	N	N	N	N	N
----------	---	---	---	---	---

Массив **P** позволяет восстановить порядок обхода вершин и хранит для каждой вершины, кроме стартовой, предшествующую в обходе вершину. На первом шаге алгоритма всем элементам массива присваивается значение «пустота» (**N**). На последнем шаге алгоритма для всех доступных вершин будут заполнены значения, равные номеру предшествующей вершины в порядке обхода.

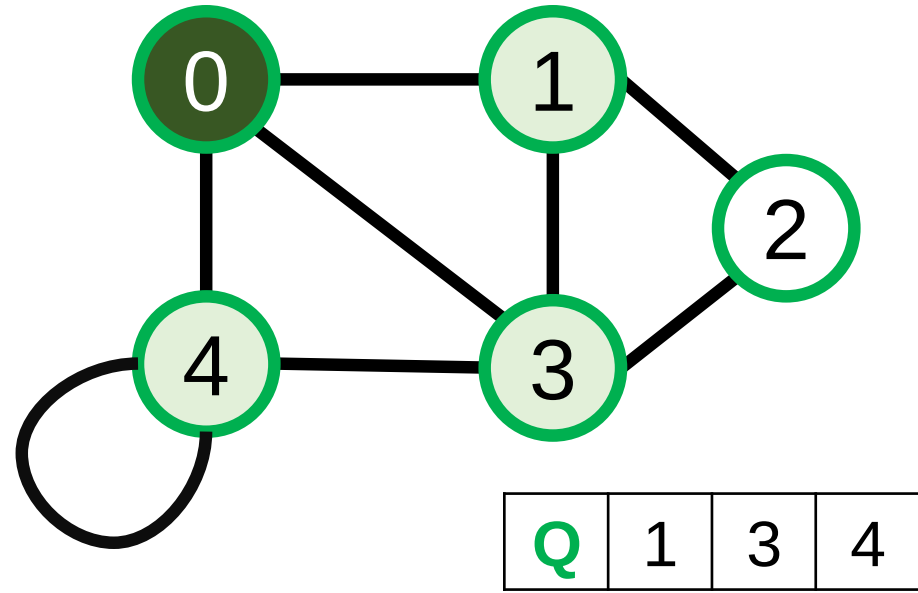
Полученный в результате работы массив P
позволяет построить так называемое BFS-дерево.



C	G	W	W	W	W
---	---	---	---	---	---

D	0	I	I	I	I
---	---	---	---	---	---

P	N	N	N	N	N
---	---	---	---	---	---



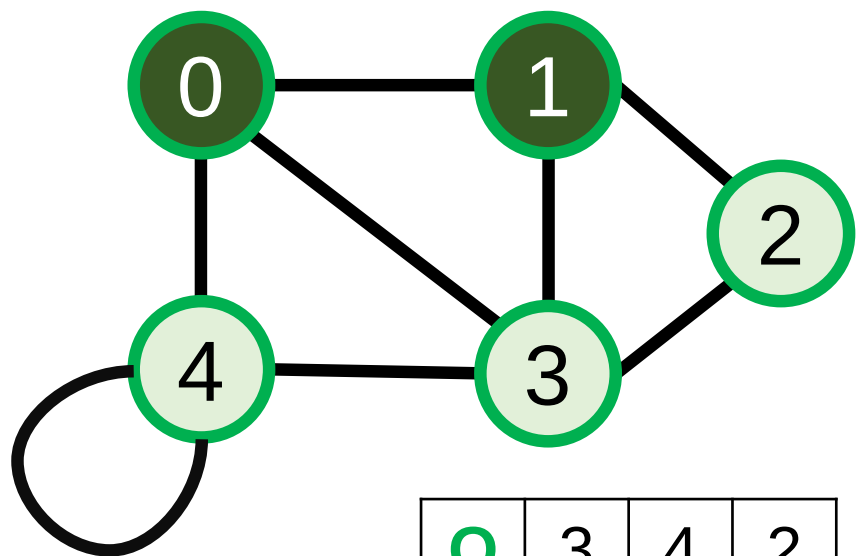
C	B	G	W	G	G
---	---	---	---	---	---

D	0	1	I	1	1
---	---	---	---	---	---

P	N	0	N	0	0
---	---	---	---	---	---

Алгоритм BFS сводится к следующей последовательности шагов.

1. Инициализировать массивы **C**, **D**, **P**. Стартовую вершину **s** поместить в очередь **Q**. и окрасить в серый цвет: $C[s] = G$. Для стартовой вершины установить расстояние, равное нулю: $D[s] = 0$.
2. Если очередь **Q** пуста, то работа алгоритма завершена, в противном случае перейти к следующему шагу.
3. Выбрать из очереди **Q** вершину **k** и окрасить ее в черный цвет: $C[k] = B$.
4. Построить множества **J** вершин белого цвета смежных вершине **k**. Если таких вершин нет, то перейти к шагу 2, иначе – к следующему шагу.
5. Каждую вершину **j** из множества **J** поместить в очередь **Q**. Обычно (но не обязательно) в очередь вершины помещаются в порядке возрастания номеров.
6. Каждую вершину **j** из множества **J** окрасить в серый цвет: $C[j] = G$.
7. Для каждой вершины **j** из множества **J** вычислить расстояние: $D[j] = D[k] + 1$.
8. Для каждой вершины **j** из множества **J** указать предшествующую вершину: $P[j] = k$.
9. Перейти к шагу 3.

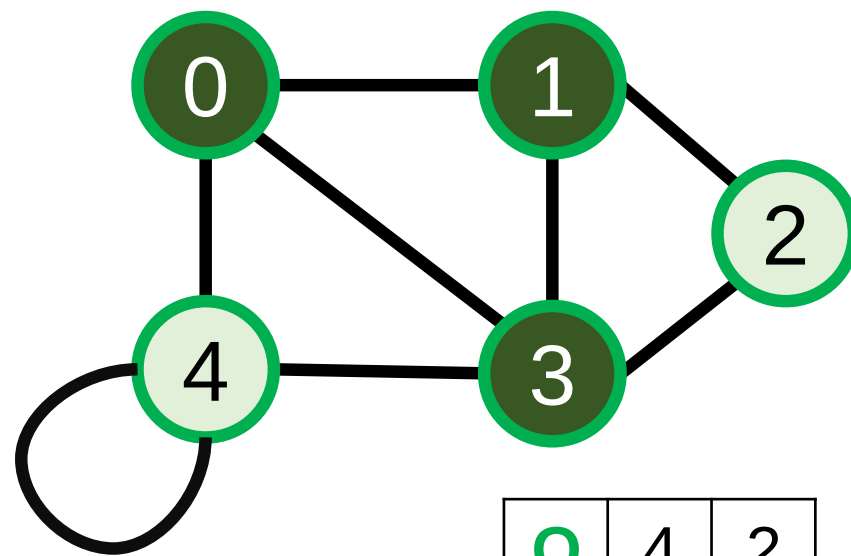


Q	3	4	2
---	---	---	---

C	B	B	G	G	G
---	---	---	---	---	---

D	0	1	2	1	1
---	---	---	---	---	---

P	N	0	1	0	0
---	---	---	---	---	---

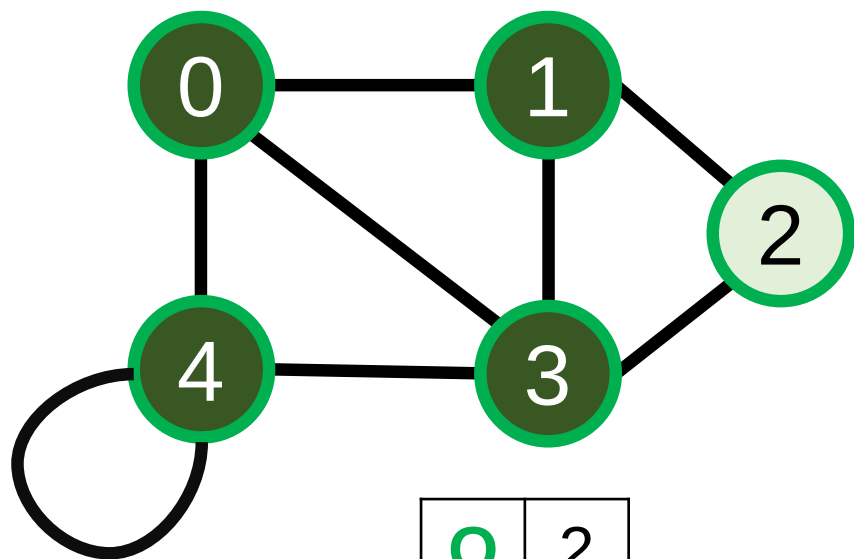


Q	4	2
---	---	---

C	B	B	G	B	G
---	---	---	---	---	---

D	0	1	2	1	1
---	---	---	---	---	---

P	N	0	1	0	0
---	---	---	---	---	---

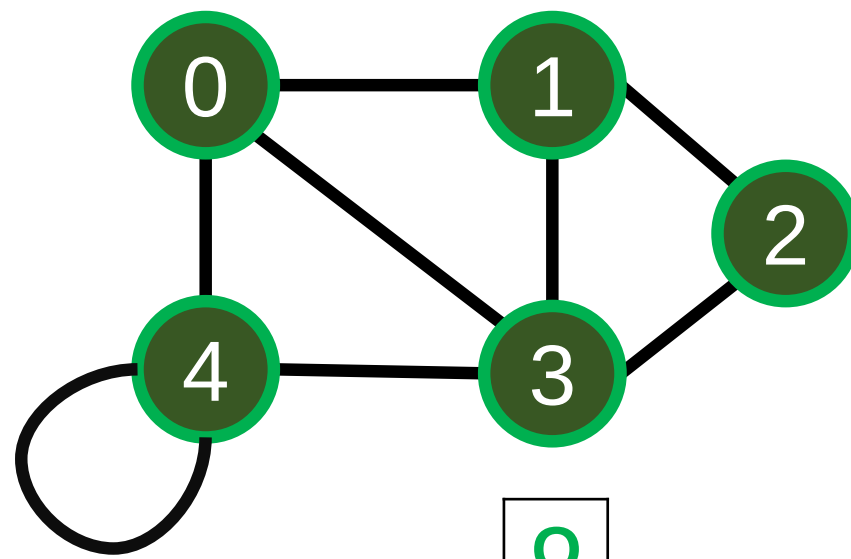


Q	2
---	---

C	B	B	G	B	B
---	---	---	---	---	---

D	0	1	2	1	1
---	---	---	---	---	---

P	N	0	1	0	0
---	---	---	---	---	---



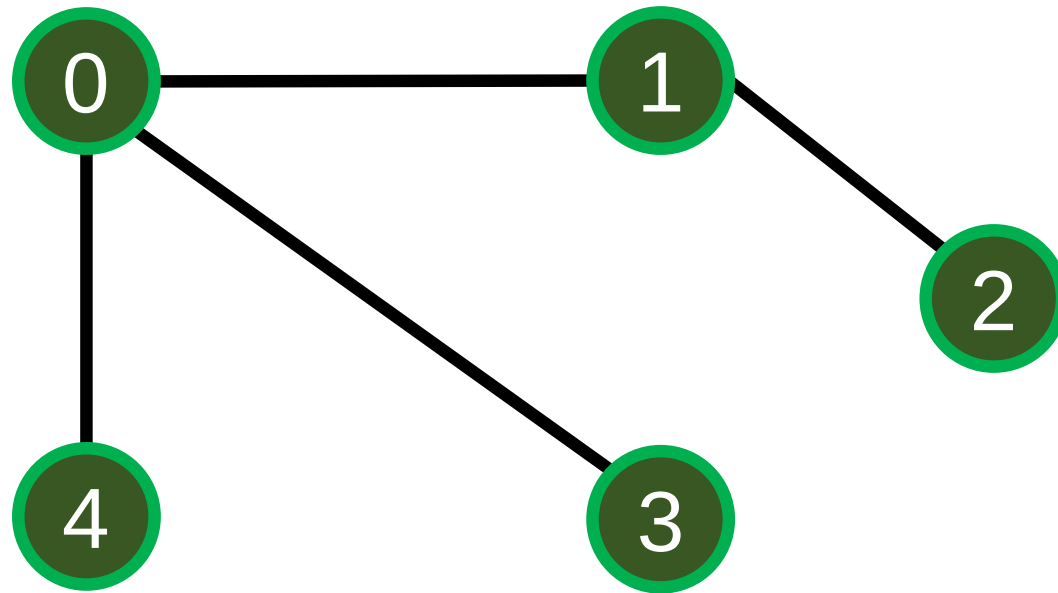
Q

C	B	B	B	B	B
---	---	---	---	---	---

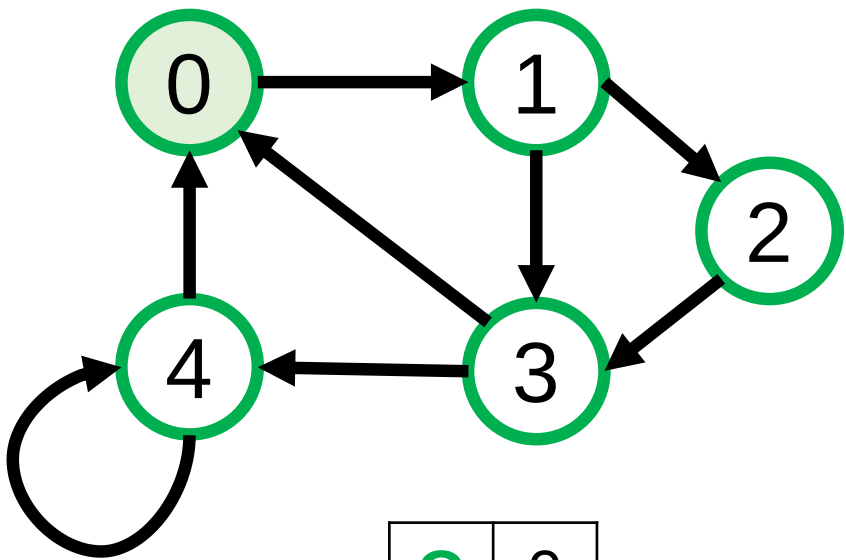
D	0	1	2	1	1
---	---	---	---	---	---

P	N	0	1	0	0
---	---	---	---	---	---

BFS-дерево – это дерево, множество вершин которого является подмножеством вершин исходного графа, связанных дугами в порядке их посещения (в соответствии с массивом **P**), а корнем – стартовая вершина.



N	0	1	0	0
---	---	---	---	---

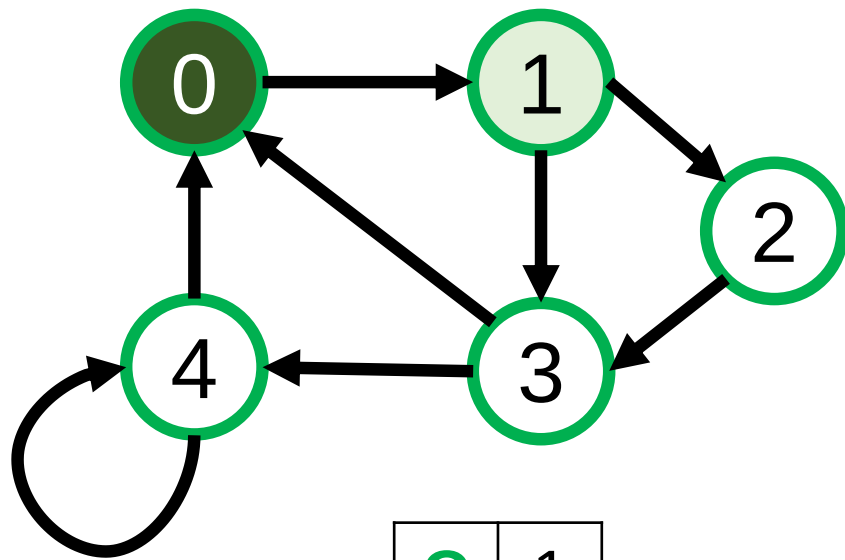


Q	0
---	---

C	G	W	W	W	W
---	---	---	---	---	---

D	0	I	I	I	I
---	---	---	---	---	---

P	N	N	N	N	N
---	---	---	---	---	---

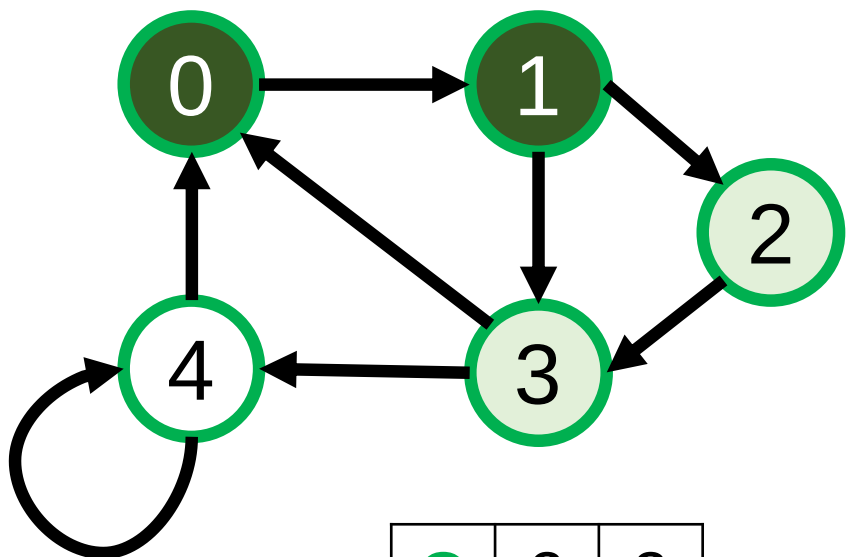


Q	1
---	---

C	B	G	W	W	W
---	---	---	---	---	---

D	0	1	I	I	I
---	---	---	---	---	---

P	N	0	N	N	N
---	---	---	---	---	---

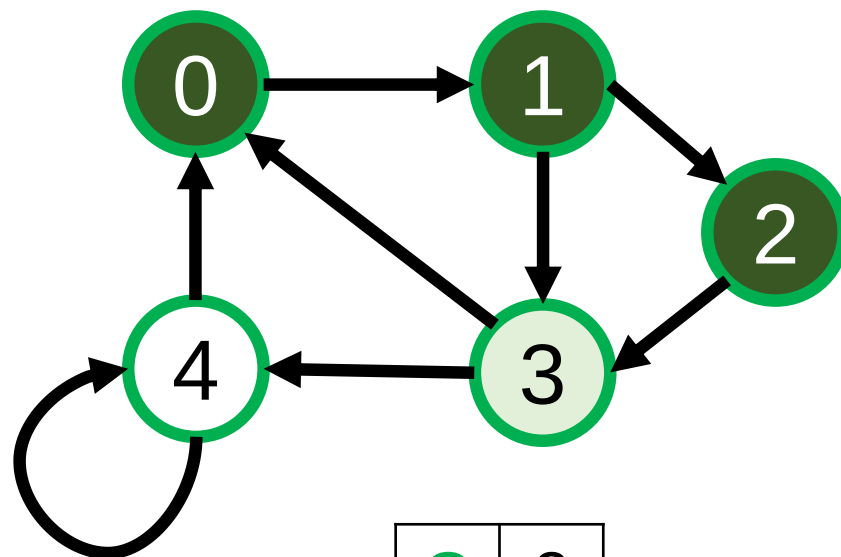


Q	2	3
---	---	---

C	B	B	G	G	W
---	---	---	---	---	---

D	0	1	2	2	I
---	---	---	---	---	---

P	N	0	1	1	N
---	---	---	---	---	---

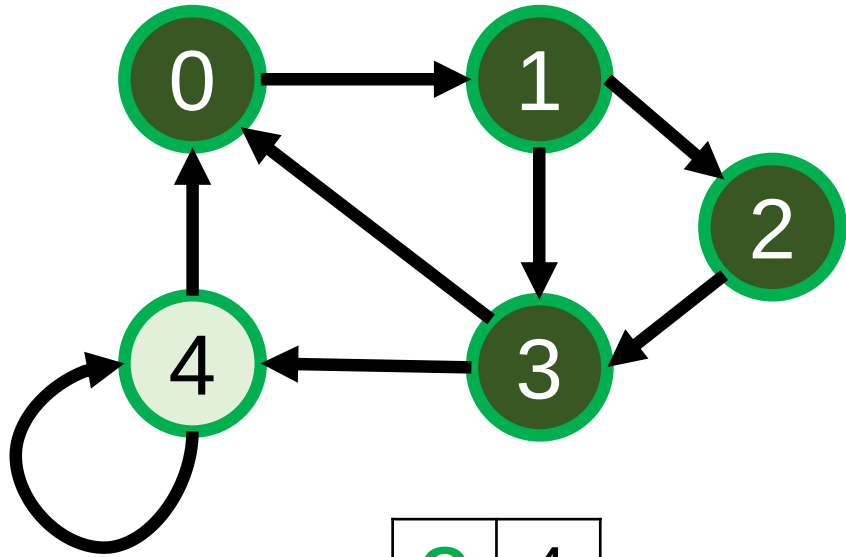


Q	3
---	---

C	B	B	B	G	W
---	---	---	---	---	---

D	0	1	2	2	I
---	---	---	---	---	---

P	N	0	1	1	N
---	---	---	---	---	---

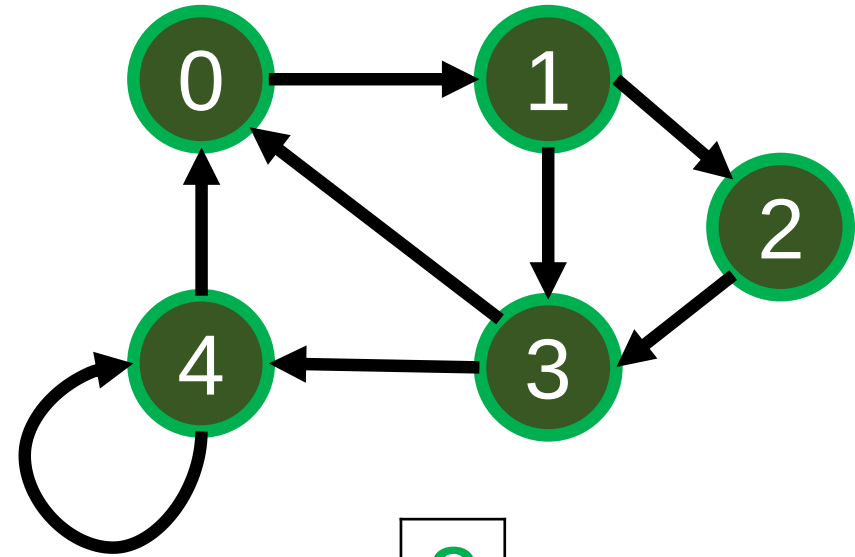


Q	4
---	---

C	B	B	B	B	W
---	---	---	---	---	---

D	0	1	2	2	3
---	---	---	---	---	---

P	N	0	1	1	3
---	---	---	---	---	---

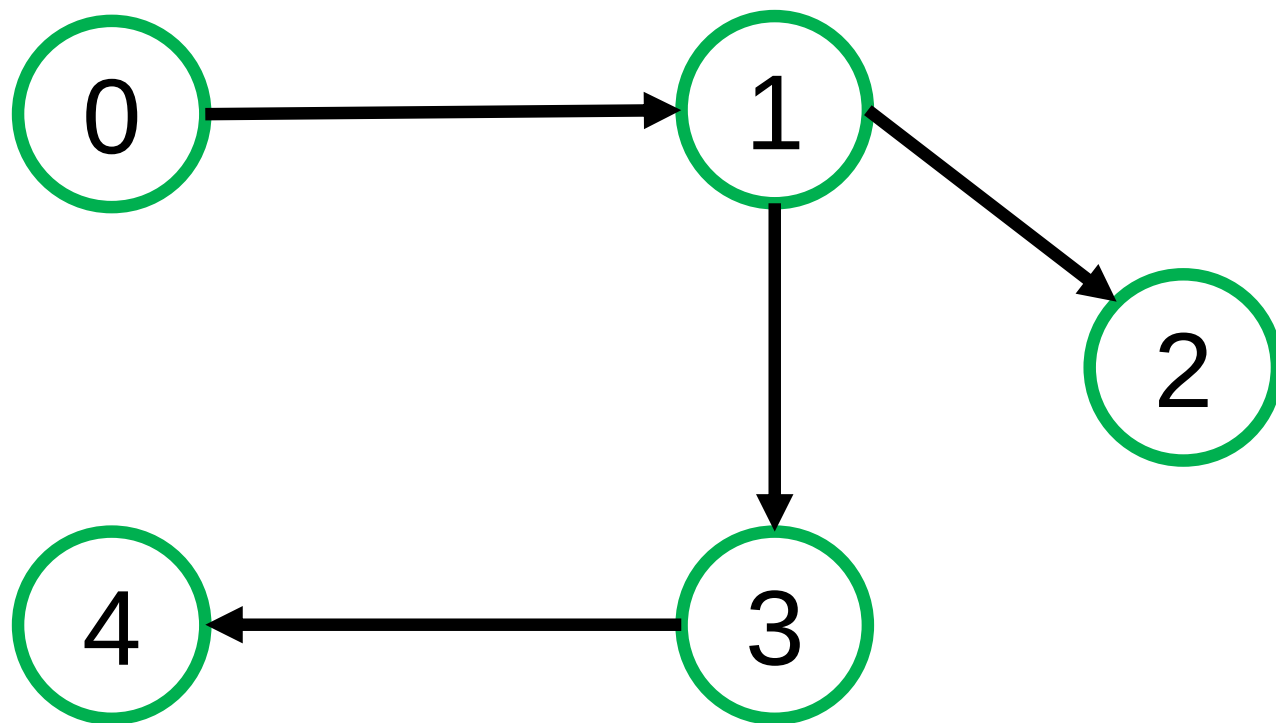


Q

C	B	B	B	B	B
---	---	---	---	---	---

D	0	1	2	2	3
---	---	---	---	---	---

P	N	0	1	1	3
---	---	---	---	---	---

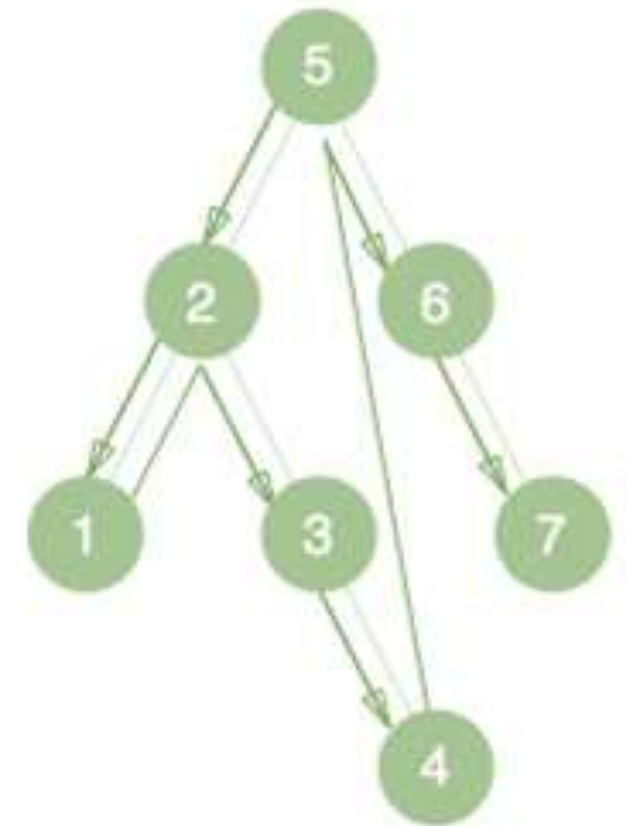


N	0	1	1	3
---	---	---	---	---

Алгоритм поиска в глубину *DFS (Depth-first search).*

Как и для поиска в ширину, задается стартовая вершина. Алгоритм описывается следующим образом: для каждой не пройденной вершины, начиная со стартовой, необходимо найти все смежные вершины и повторить поиск для каждой.

Назначение и размерность массивов **C** (массив окраски вершин) и **P** (массив предшествующих вершин) такие же, как и в алгоритме BFS. В массиве **D** для каждой вершины записывается время обнаружения (шаг окраски в серый цвет). Массив **F** предназначен для хранения времени фиксации (шага окраски в черный цвет) вершины. Кроме того, используется переменная **t**, текущее значение которой – номер шага алгоритма.



Depth-first traversal

Алгоритм поиска в глубину *DFS (Depth-first search).*

В основе алгоритма DFS лежит рекурсивная процедура **Visit**, имеющая один входной параметр **k** – вершину графа.

Опишем пошагово процедуру **Visit**.

1. Принять параметр **k** – вершину графа.
2. Вершину **k** окрасить в серый цвет: $C[k] = G$.
3. Увеличить номер шага: $t = t + 1$.
4. Подсчитать расстояние до вершины: $D[k] = t$. Расстояние до вершины в алгоритме DFS совпадает с номером шага, на котором эта вершина была обнаружена (окрашена в серый цвет).
5. Построить множества **J** вершин белого цвета, смежных вершине **k**. Если таких вершин нет, то перейти к шагу 8.

Алгоритм поиска в глубину *DFS (Depth-first search).*

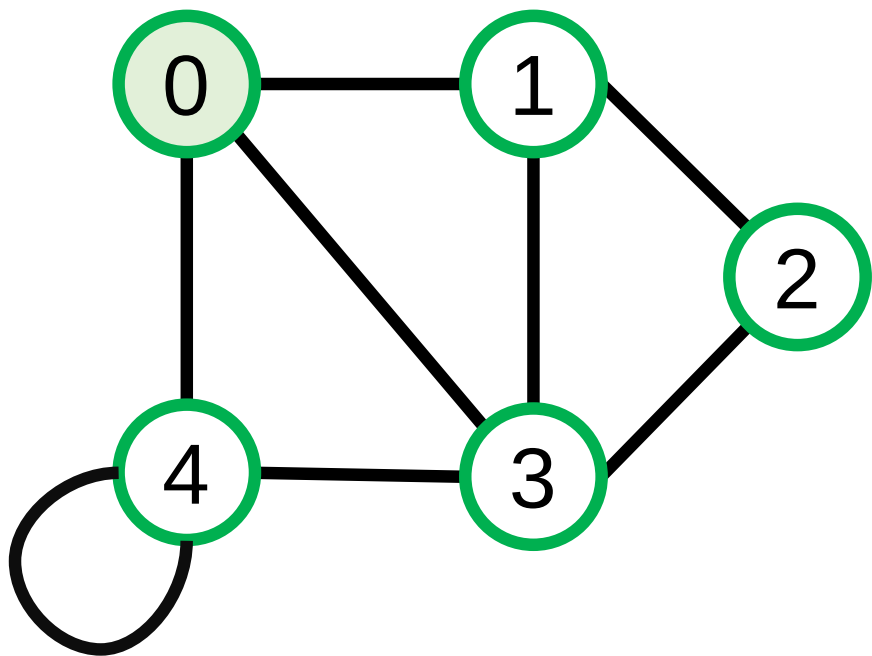
продолжение пошагово описания процедуры Visit.

6. Для каждой вершины j из множества J указать предшествующую вершину: $P[j] = k$.
7. Для каждой вершины j из множества J выполнить процедуру **Visit**.
8. Вершину k окрасить в черный цвет: $C[k] = B$.
9. Увеличить номер шага: $t = t + 1$.
10. Отметить время фиксации вершины: $F[k] = t$.

Алгоритм поиска в глубину *DFS (Depth-first search).*

Если задана стартовая вершина **s**, то алгоритм DFS теперь можно свести к следующим двум шагам.

1. Инициализировать массивы: **C** (все вершины белого цвета), **D** (все расстояния равны бесконечности), **P** (все элементы заполнены символом «пустота»). Установить номер шага: **t = 0**.
2. Выполнить процедуру **Visit** для вершины **s**.



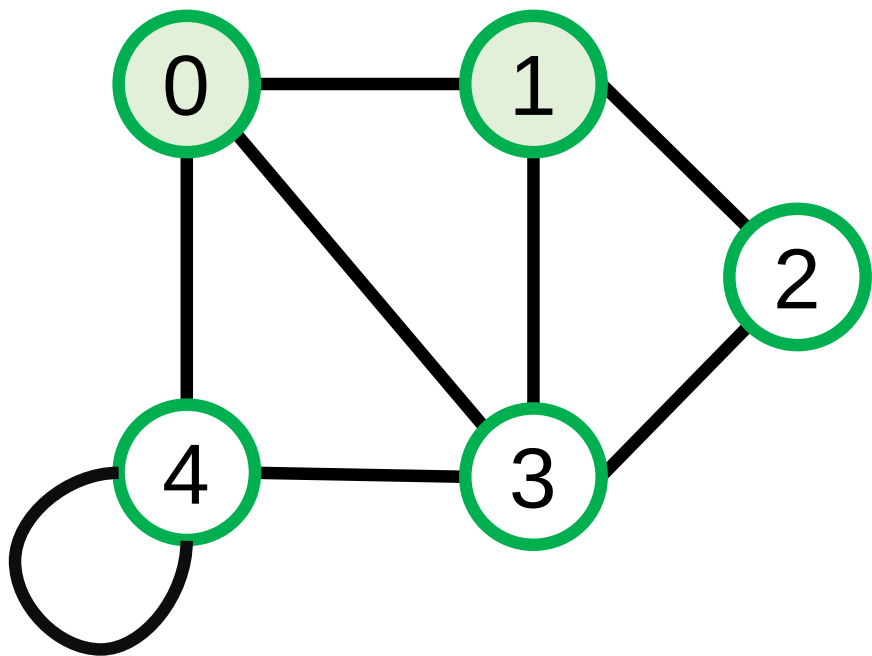
t = 1

C	G	W	W	W	W
----------	---	---	---	---	---

D	1	I	I	I	I
----------	---	---	---	---	---

P	N	N	N	N	N
----------	---	---	---	---	---

F	0	0	0	0	0
----------	---	---	---	---	---



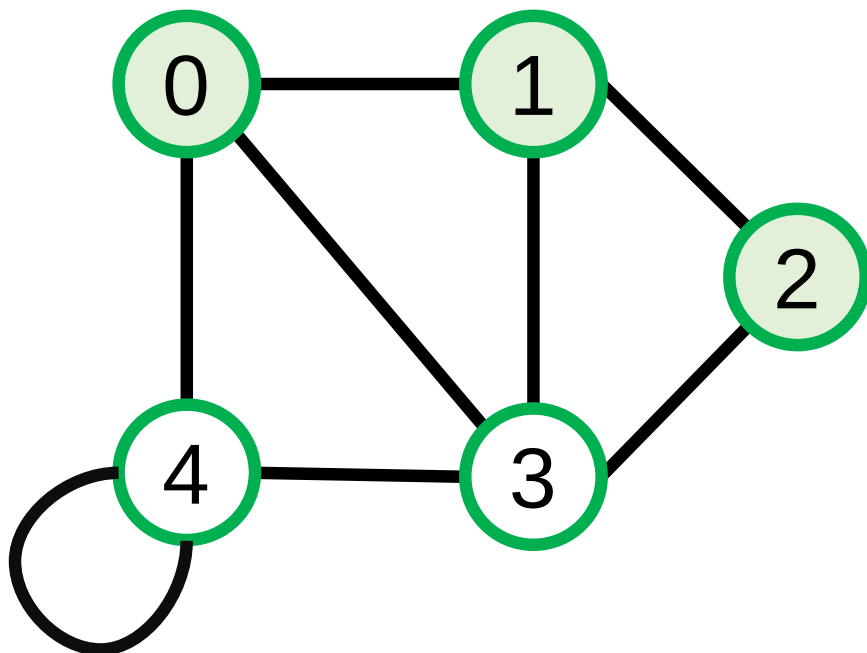
t = 2

C	G	G	W	W	W
----------	---	---	---	---	---

D	1	2	I	I	I
----------	---	---	---	---	---

P	N	0	N	N	N
----------	---	---	---	---	---

F	0	0	0	0	0
----------	---	---	---	---	---



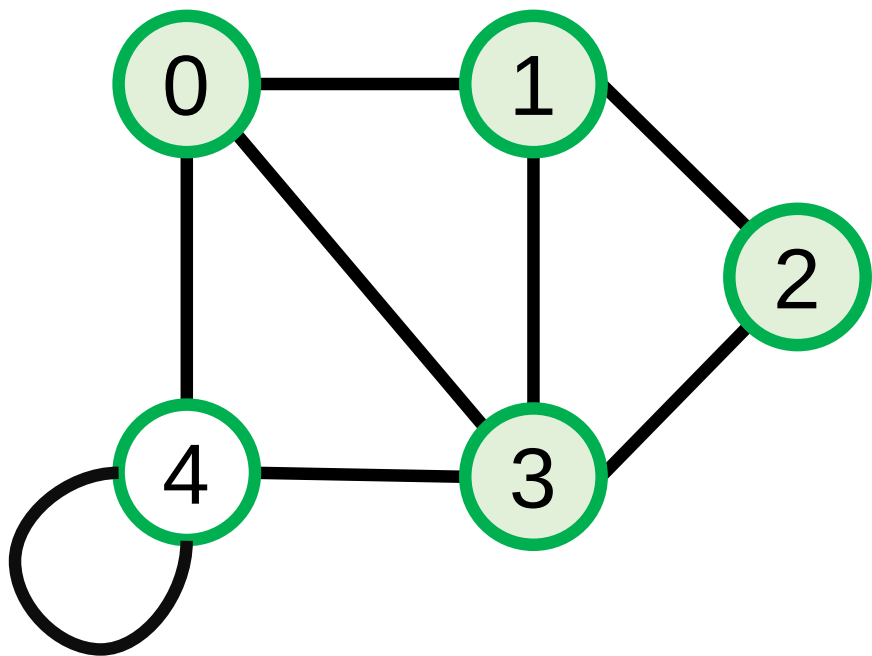
t = 3

C	G	G	G	W	W
----------	---	---	---	---	---

D	1	2	3	I	I
----------	---	---	---	---	---

P	N	0	1	N	N
----------	---	---	---	---	---

F	0	0	0	0	0
----------	---	---	---	---	---



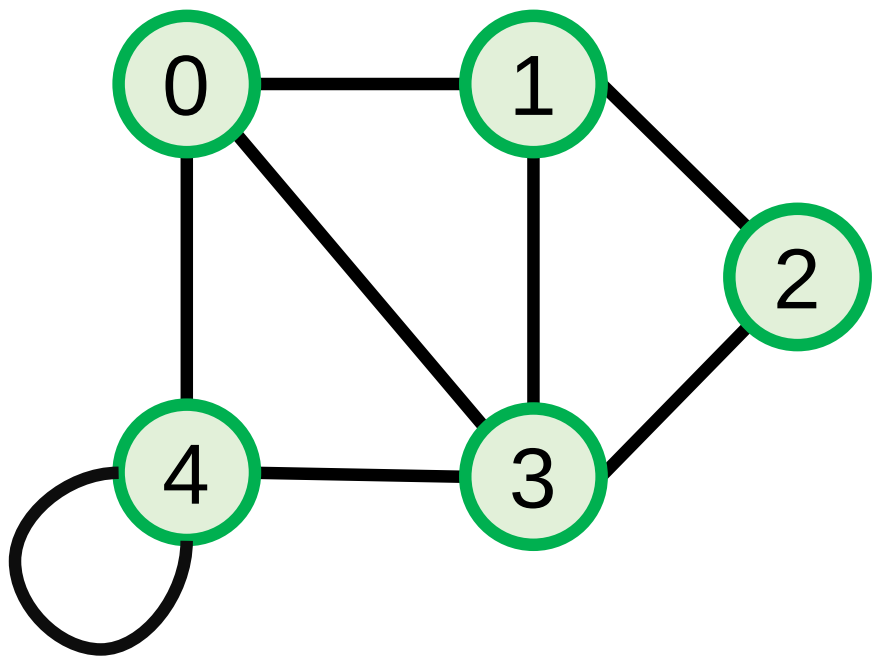
t = 4

C	G	G	G	G	W
----------	---	---	---	---	---

D	1	2	3	4	I
----------	---	---	---	---	---

P	N	0	1	2	N
----------	---	---	---	---	---

F	0	0	0	0	0
----------	---	---	---	---	---



t = 5

C

G

G

G

G

G

D

1

2

3

4

5

P

N

0

1

2

3

F

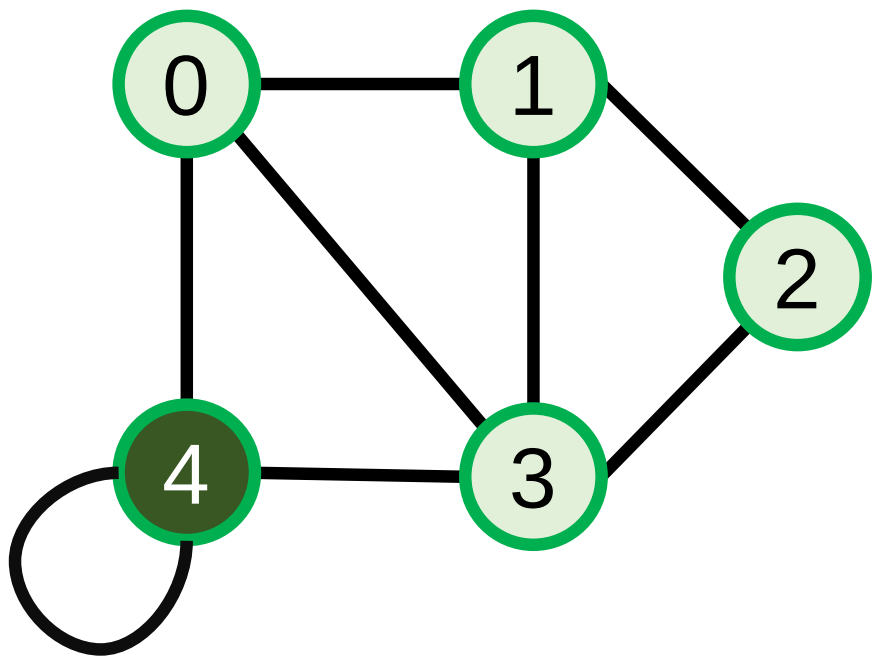
0

0

0

0

0



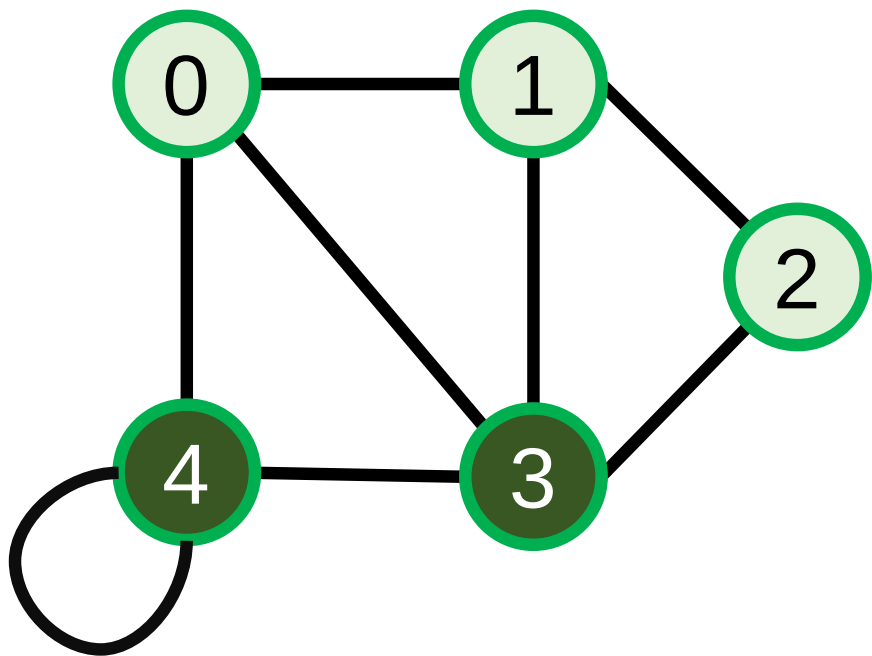
t = 6

C	G	G	G	G	B
----------	---	---	---	---	---

D	1	2	3	4	5
----------	---	---	---	---	---

P	N	0	1	2	3
----------	---	---	---	---	---

F	0	0	0	0	6
----------	---	---	---	---	---



t = 7

C

G

G

G

B

B

D

1

2

3

4

5

P

N

0

1

2

3

F

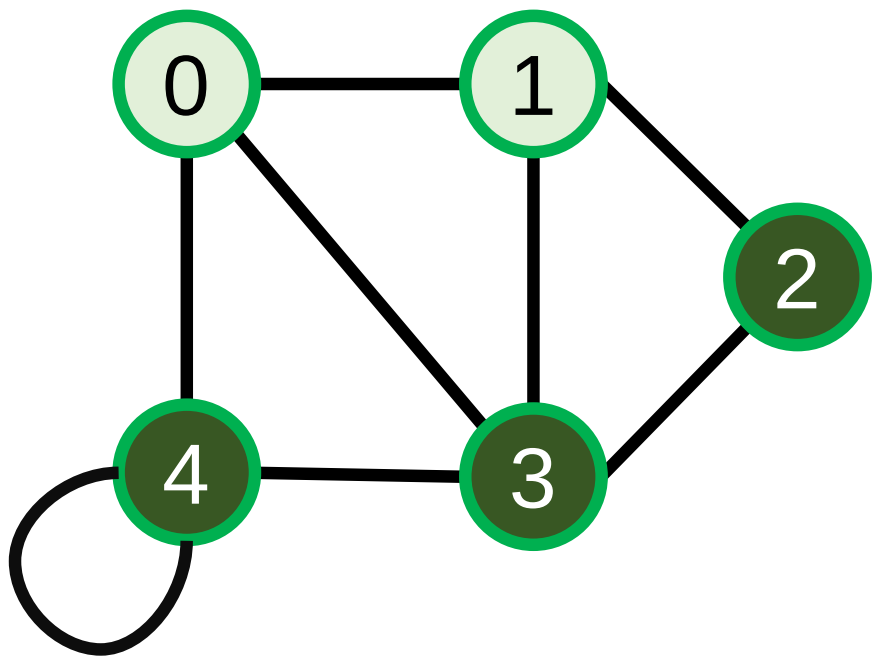
0

0

0

7

6



t = 8

C

G

G

B

B

B

D

1

2

3

4

5

P

N

0

1

2

3

F

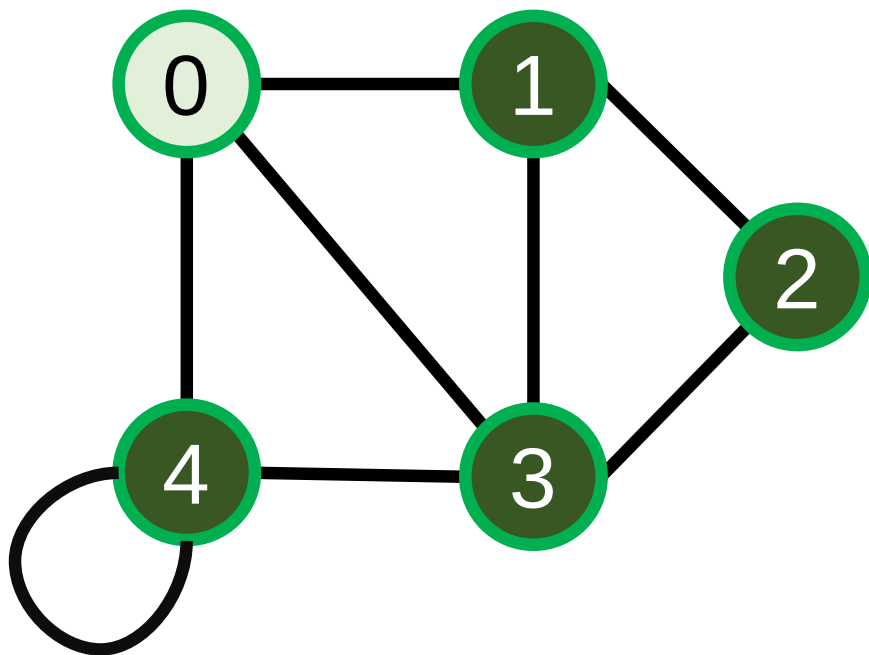
0

0

8

7

6



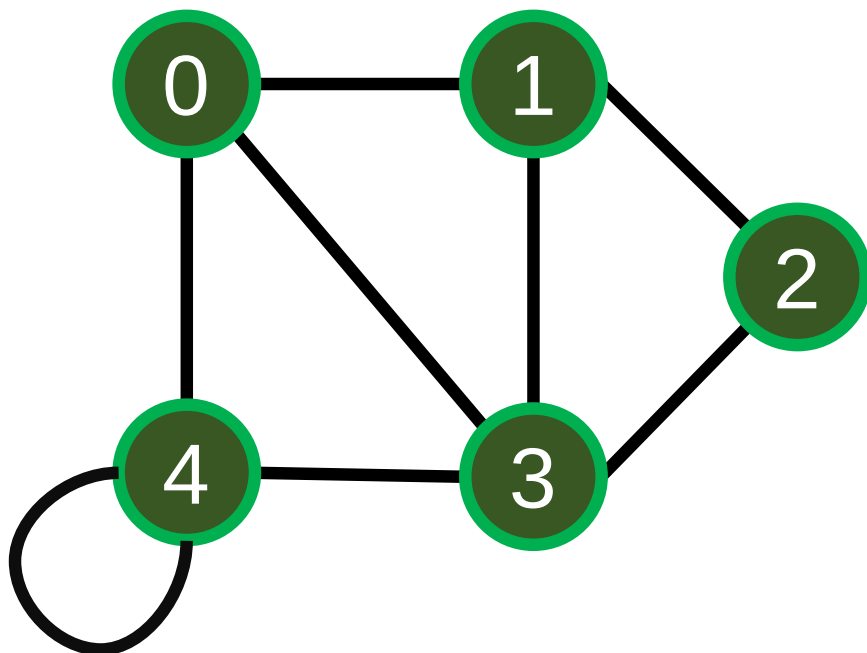
t = 9

C	G	B	B	B	B
----------	---	---	---	---	---

D	1	2	3	4	5
----------	---	---	---	---	---

P	N	0	1	2	3
----------	---	---	---	---	---

F	0	9	8	7	6
----------	---	---	---	---	---



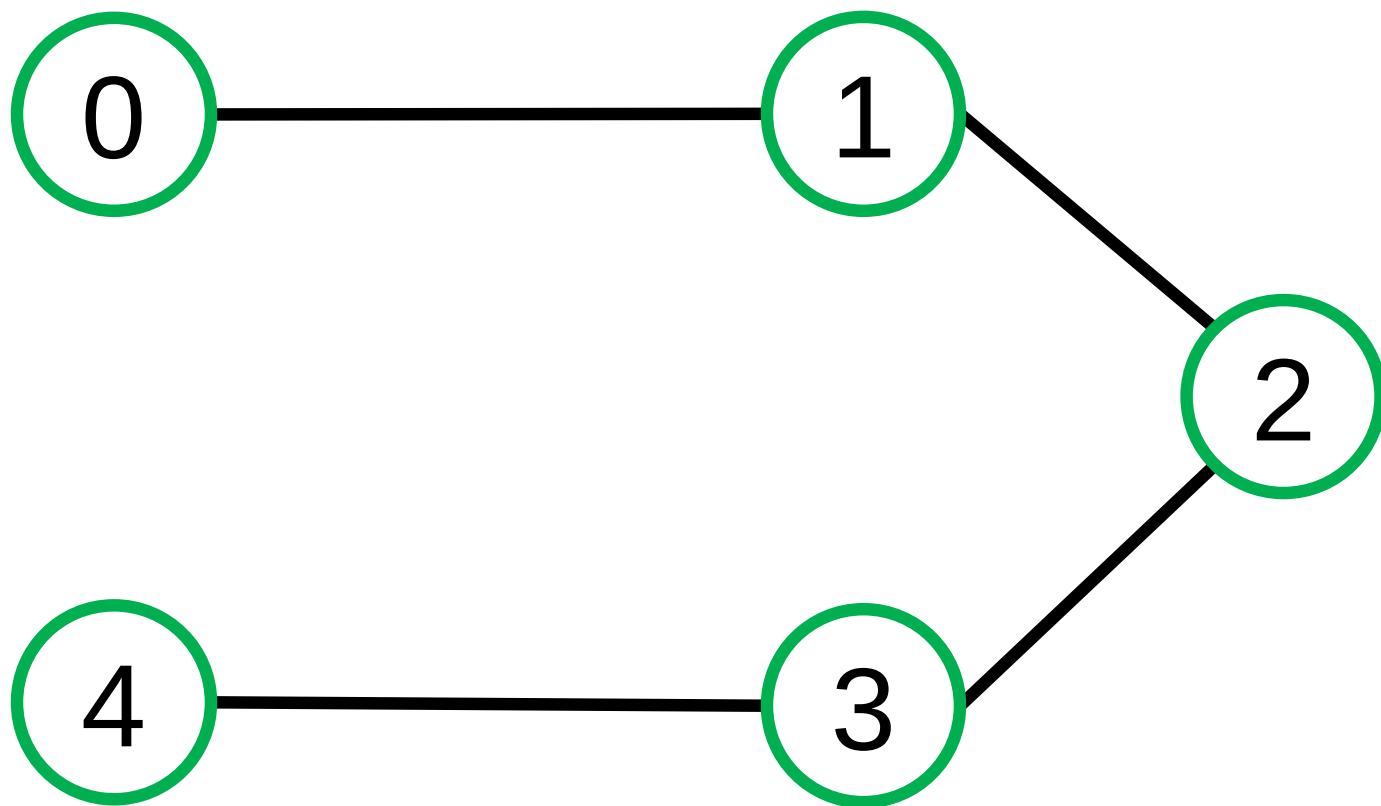
t = 10

C	B	B	B	B	B
----------	---	---	---	---	---

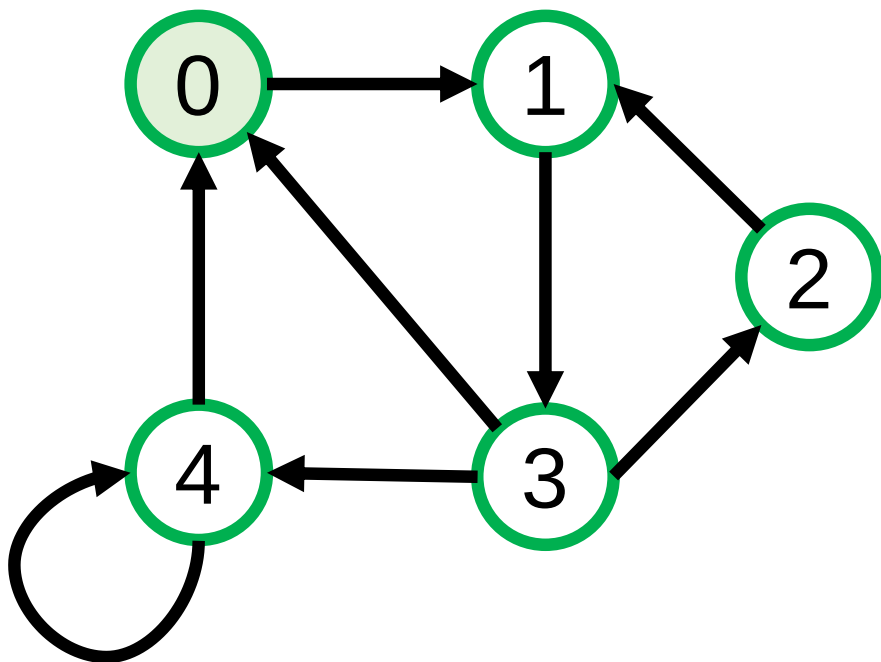
D	1	2	3	4	5
----------	---	---	---	---	---

P	N	0	1	2	3
----------	---	---	---	---	---

F	1	9	8	7	6
	0				



P	N	0	1	2	3
---	---	---	---	---	---



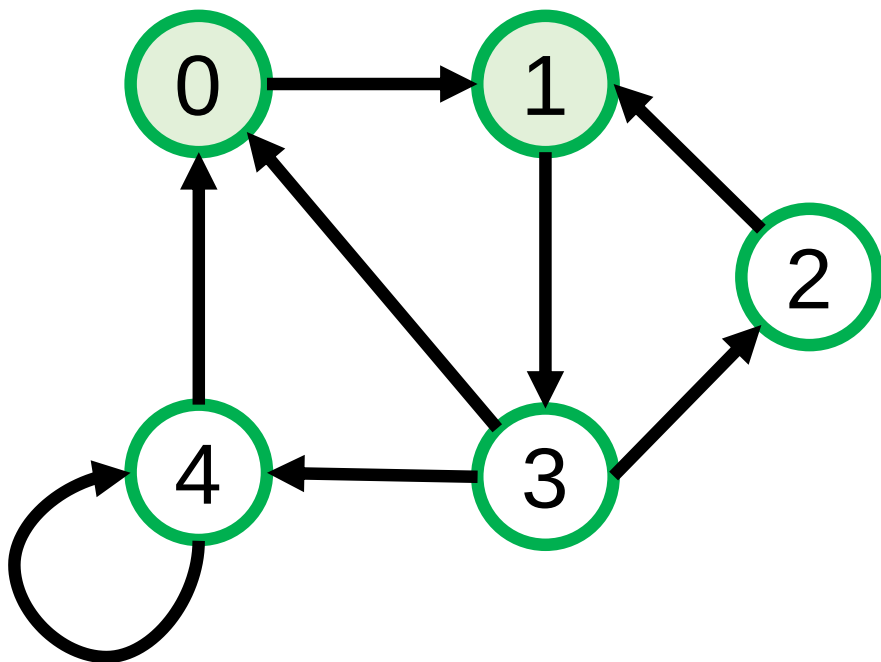
t = 1

C	G	W	W	W	W
----------	---	---	---	---	---

D	1	I	I	I	I
----------	---	---	---	---	---

P	N	N	N	N	N
----------	---	---	---	---	---

F	0	0	0	0	0
----------	---	---	---	---	---



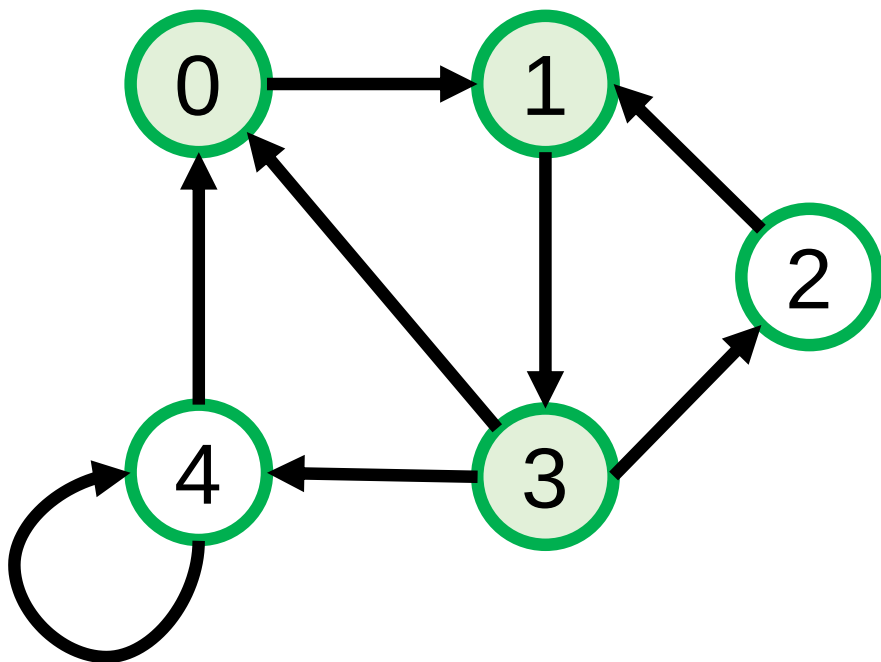
t = 2

C	G	G	W	W	W
----------	---	---	---	---	---

D	1	2	I	I	I
----------	---	---	---	---	---

P	N	0	N	N	N
----------	---	---	---	---	---

F	0	0	0	0	0
----------	---	---	---	---	---



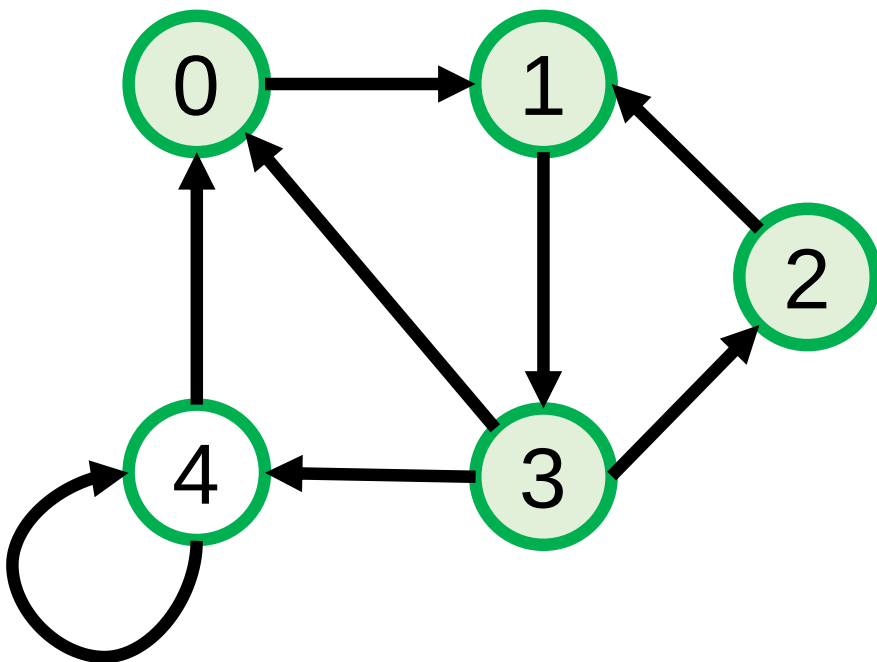
t = 3

C	G	G	W	G	W
----------	---	---	---	---	---

D	1	2	I	3	I
----------	---	---	---	---	---

P	N	0	N	1	N
----------	---	---	---	---	---

F	0	0	0	0	0
----------	---	---	---	---	---



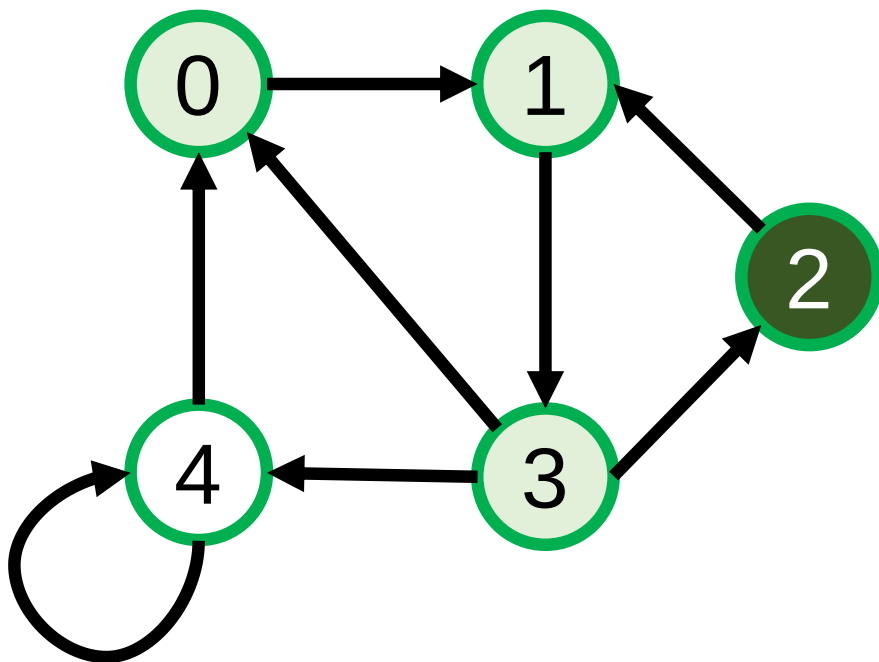
t = 4

C	G	G	G	G	W
----------	---	---	---	---	---

D	1	2	4	3	I
----------	---	---	---	---	---

P	N	0	3	1	N
----------	---	---	---	---	---

F	0	0	0	0	0
----------	---	---	---	---	---



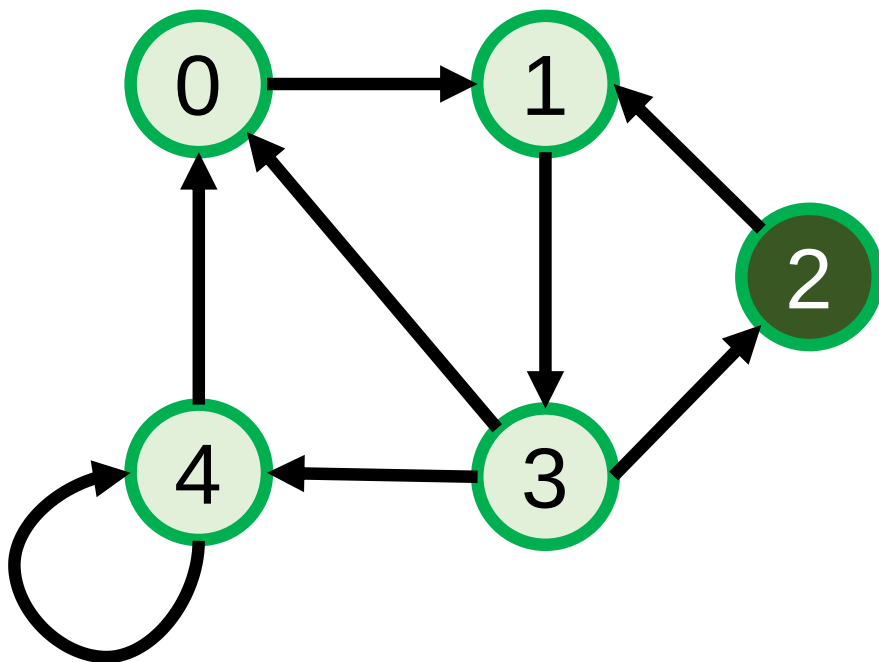
t = 5

C	G	G	B	G	W
----------	---	---	---	---	---

D	1	2	4	3	I
----------	---	---	---	---	---

P	N	0	3	1	N
----------	---	---	---	---	---

F	0	0	5	0	0
----------	---	---	---	---	---



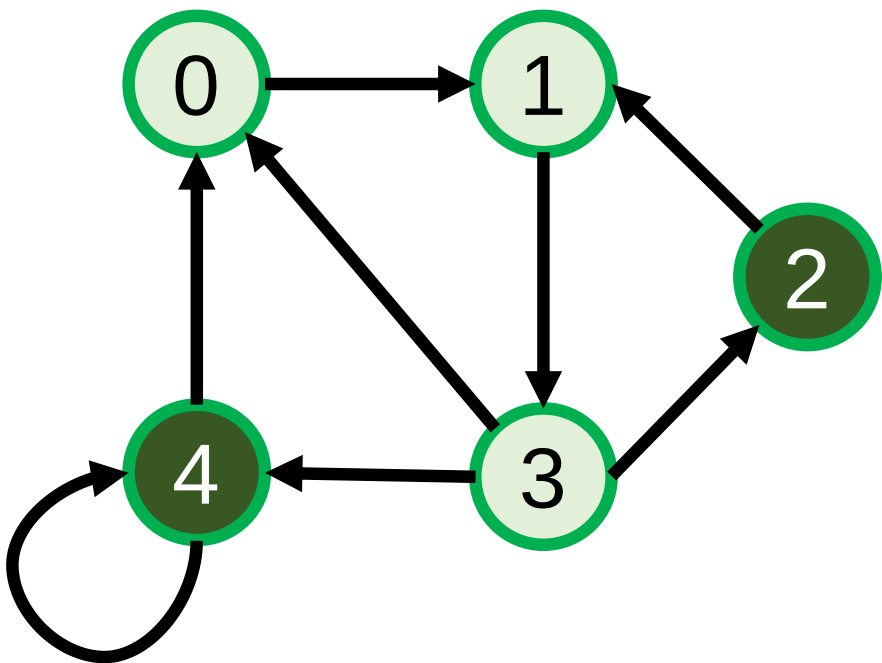
t = 6

C	G	G	B	G	G
----------	---	---	---	---	---

D	1	2	4	3	6
----------	---	---	---	---	---

P	N	0	3	1	3
----------	---	---	---	---	---

F	0	0	5	0	0
----------	---	---	---	---	---



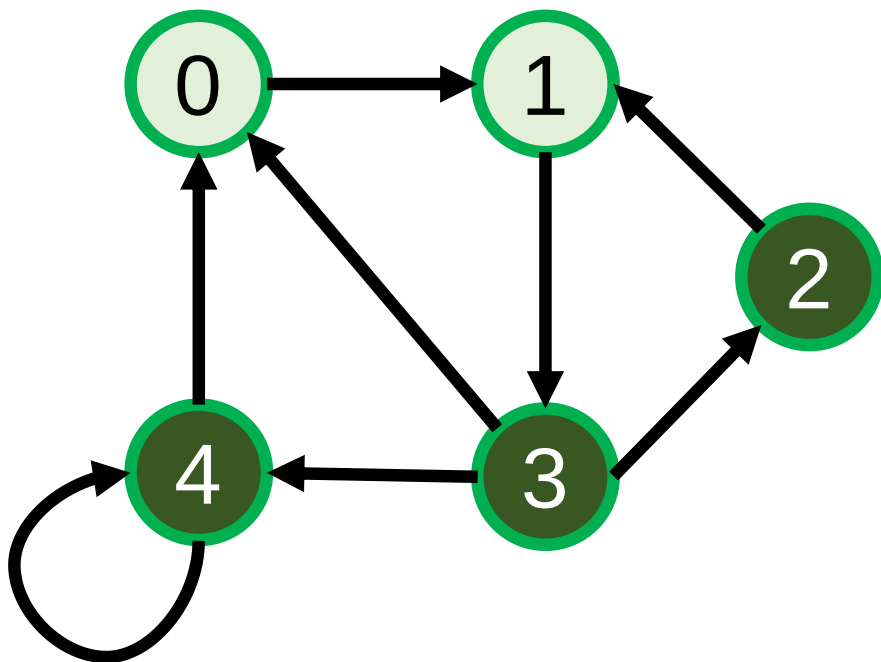
t = 7

C	G	G	B	G	B
----------	---	---	---	---	---

D	1	2	4	3	6
----------	---	---	---	---	---

P	N	0	3	1	3
----------	---	---	---	---	---

F	0	0	5	0	7
----------	---	---	---	---	---



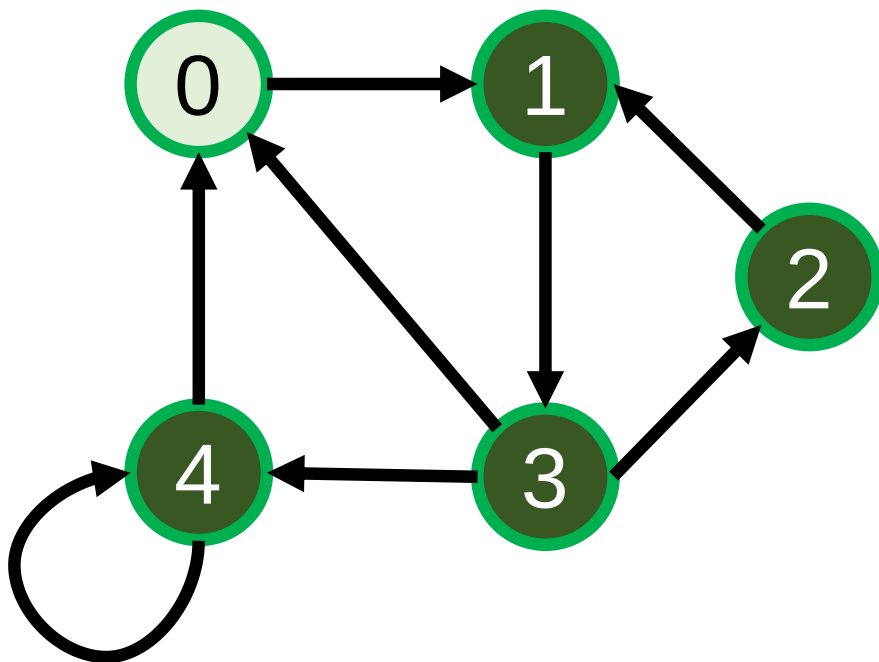
t = 8

C	G	G	B	B	B
----------	---	---	---	---	---

D	1	2	4	3	6
----------	---	---	---	---	---

P	N	0	3	1	3
----------	---	---	---	---	---

F	0	0	5	8	7
----------	---	---	---	---	---



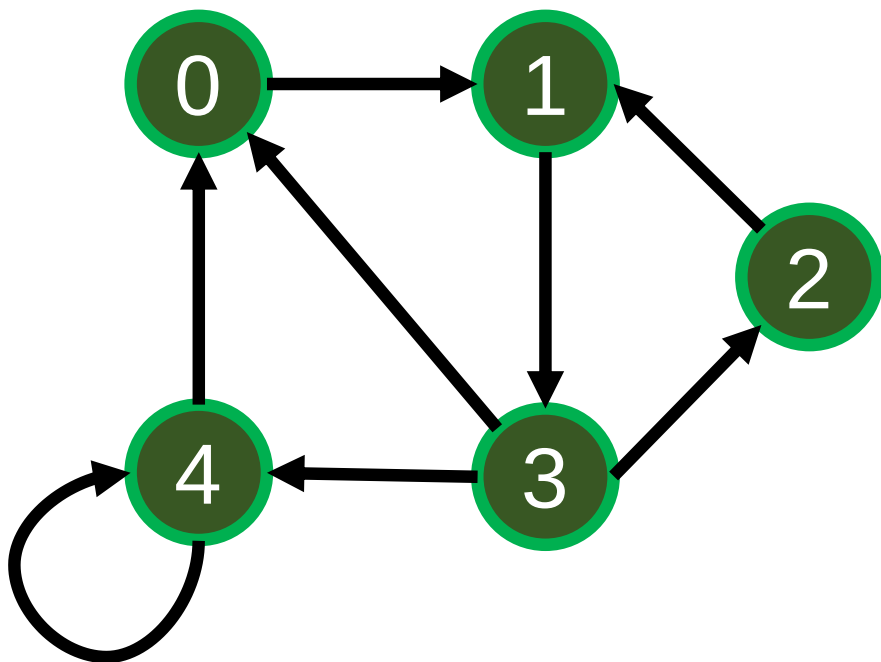
t = 9

C	G	B	B	B	B
----------	---	---	---	---	---

D	1	2	4	3	6
----------	---	---	---	---	---

P	N	0	3	1	3
----------	---	---	---	---	---

F	0	9	5	8	7
----------	---	---	---	---	---



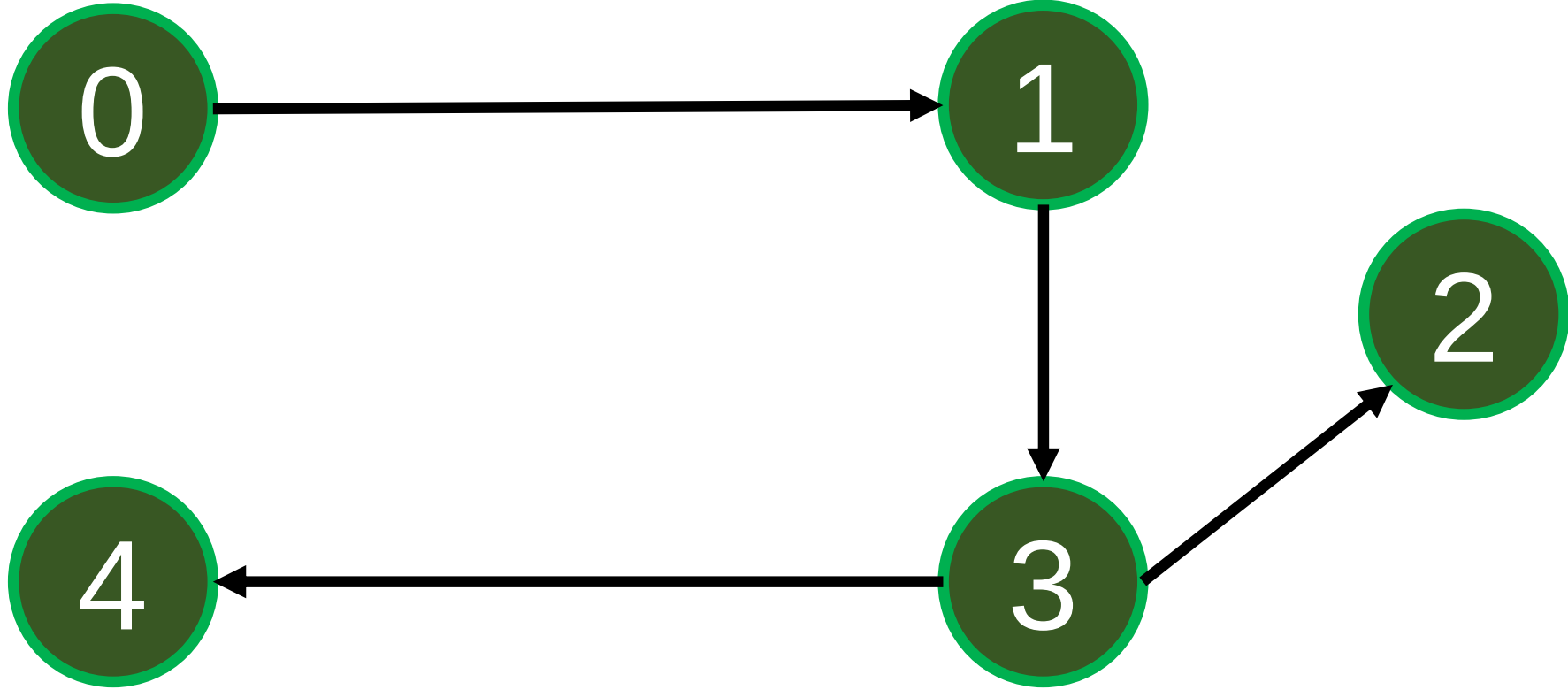
t = 10

C	B	B	B	B	B
----------	---	---	---	---	---

D	1	2	4	3	6
----------	---	---	---	---	---

P	N	0	3	1	3
----------	---	---	---	---	---

F	1	9	5	8	7
	0				



P	N	0	3	1	3
---	---	---	---	---	---

Топологическая сортировка вершин графа

Часто с помощью графов описывается некоторая последовательность связанных действий. Например, если поставить в соответствие каждой странице сайта вершину графа, а дуге – ссылку (возможность перехода) с одной страницу на другую, то такой граф будет отображать модель (карту) сайта.

Другой пример: применение графов для описания проектов. Как правило, в этом случае вершинам графа соответствуют события проекта (завершение этапа), дугам – операции. Такой граф описывает комплекс связанных операций проекта.

Если графы включают много связанных вершин, то модели, основанные на них, становятся трудно воспринимаемыми. В некоторых случаях вершины графа можно упорядочить таким образом, что вид графа становится более структурированным.

Топологическая сортировка вершин графа

Топологическая сортировка – это процедура упорядочивания вершин ориентированного графа, не имеющего циклов (ациклического графа). В результате топологической сортировки для вершин графа определяется такой порядок, что если их расположить на рисунке в соответствии с этим порядком сверху вниз, то дуги будут направлены только от верхних вершин к нижним. Обычно после выполнения топологической сортировки вершины переименовываются (перенумеровываются) в соответствии с полученным порядком. После такого переименования граф обладает свойством: начальная вершина каждой дуги имеет номер (имя) меньший, чем номер конечной вершины этой дуги.

Топологическая сортировка вершин графа

Наиболее известны два способа топологической сортировки графа: алгоритмы Демукрона и алгоритм, применяющий поиск в глубину.

Топологическая сортировка графа с помощью алгоритма поиска в глубину.

Запустить алгоритм DFS, при выходе из вершины добавляя вершину в конец списка с ответом. После окончания алгоритма список с ответом развернуть в противоположном порядке.

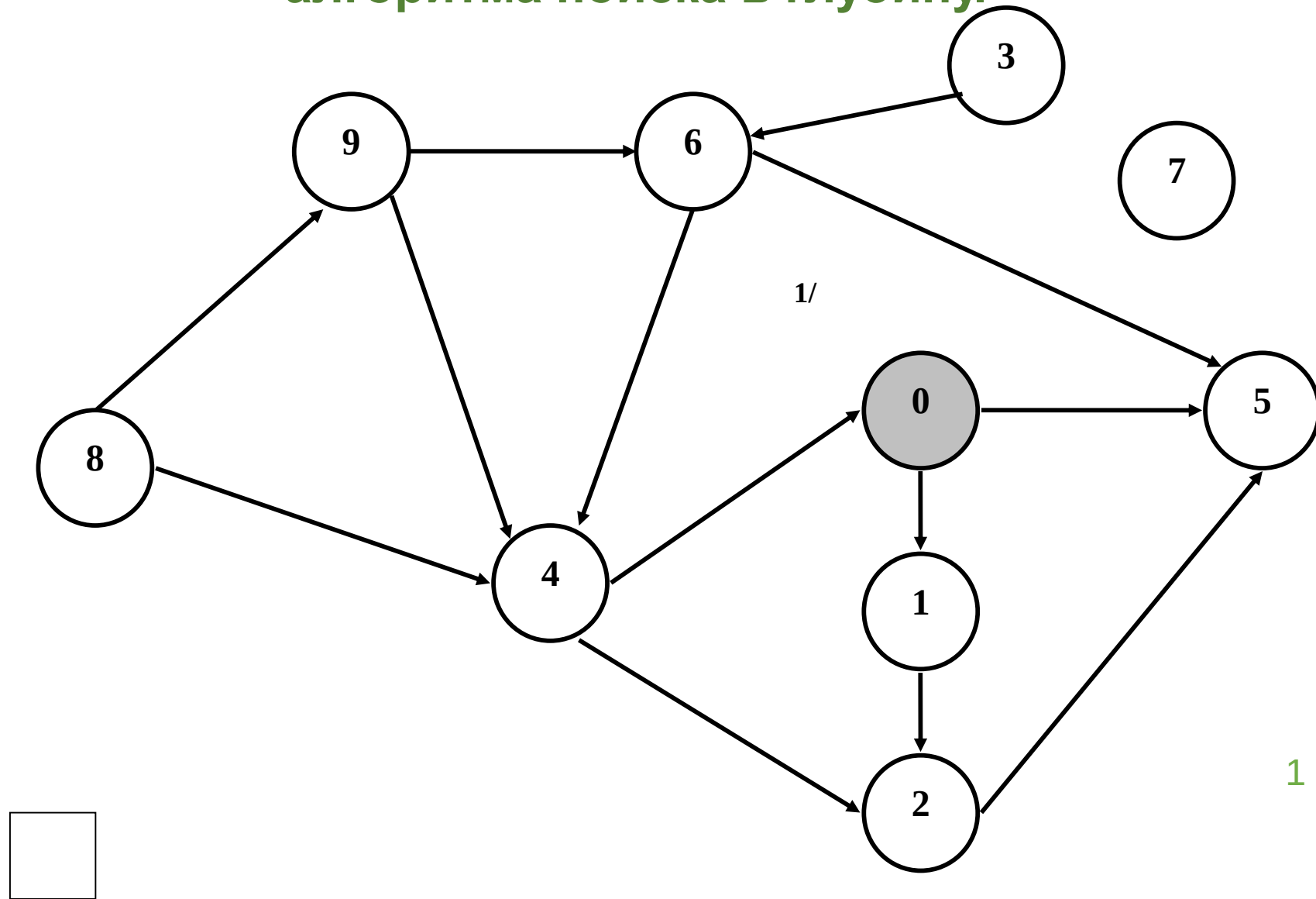
Топологическая сортировка вершин графа

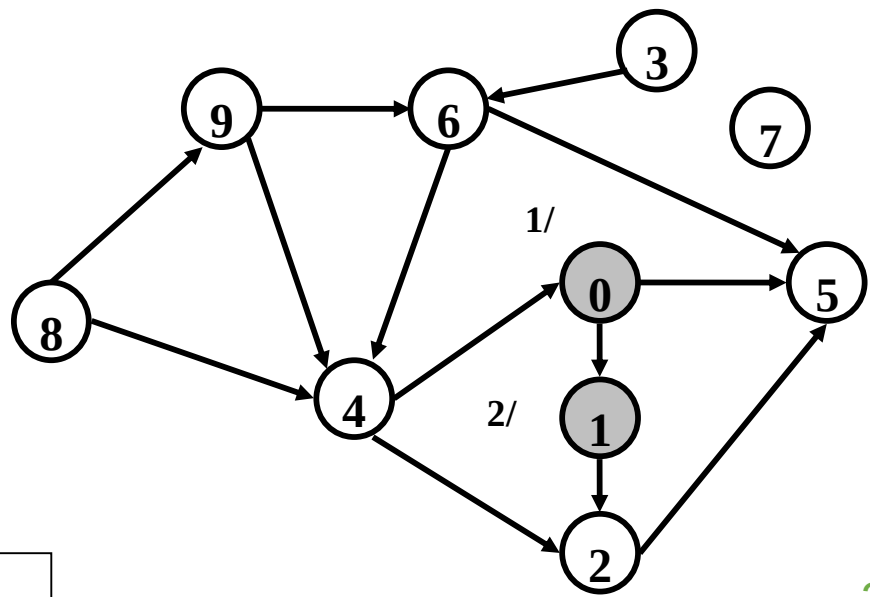
Алгоритм Демукрона вычисления порядковой функции сети

Этот метод состоит в вычислении порядковой функции сети и известен как алгоритм Демукрона. Предполагается, что вершины сети пронумерованы от 1 до n .

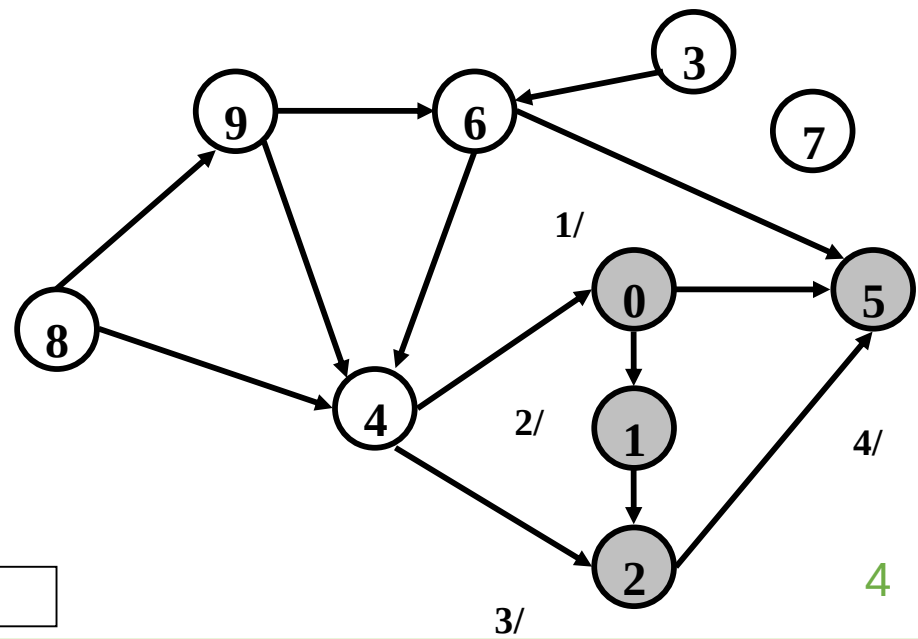
Наглядно процесс определения уровней вершин можно представить следующим образом. Нулевой уровень образуют входы сети — вершины с полустепенью захода, равной 0. Удалив из сети все вершины нулевого уровня и исходящие из них дуги, вновь получим сеть, входами которой будут вершины первого уровня исходной сети. Указанный процесс "послойного" удаления вершин следует продолжать до тех пор, пока все вершины исходной сети не будут распределены по уровням.

Пример: топологическая сортировка графа с помощью алгоритма поиска в глубину.

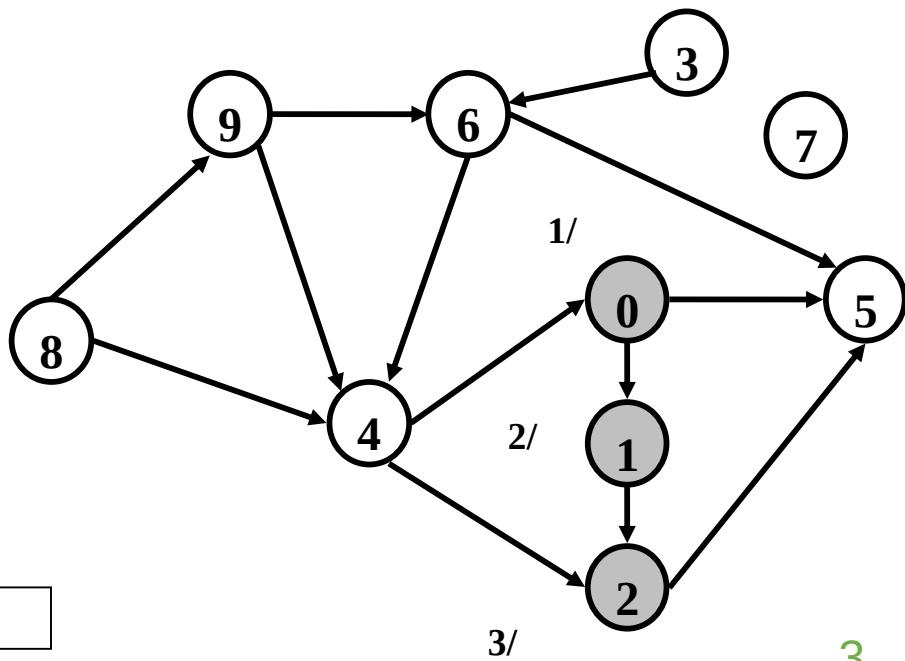




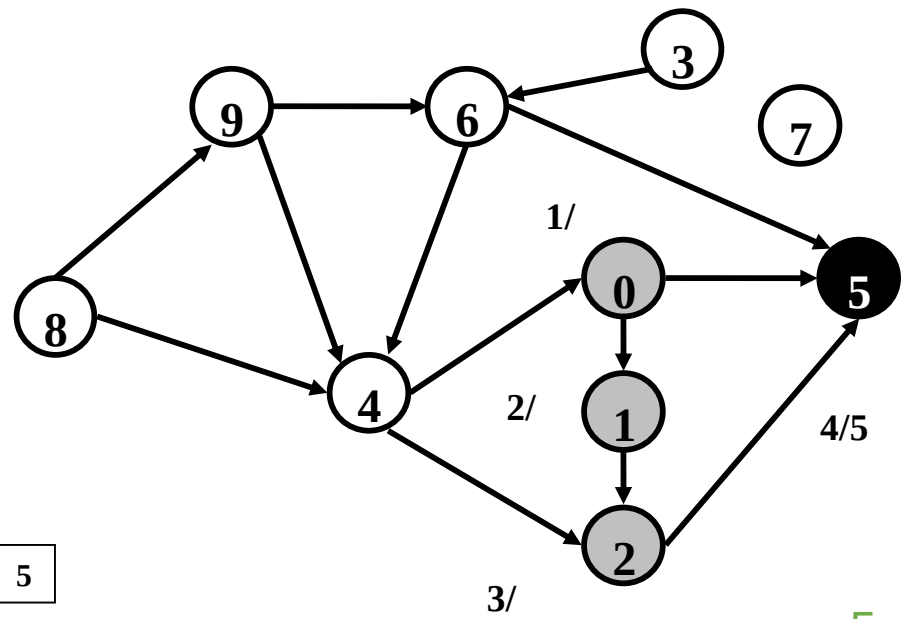
2



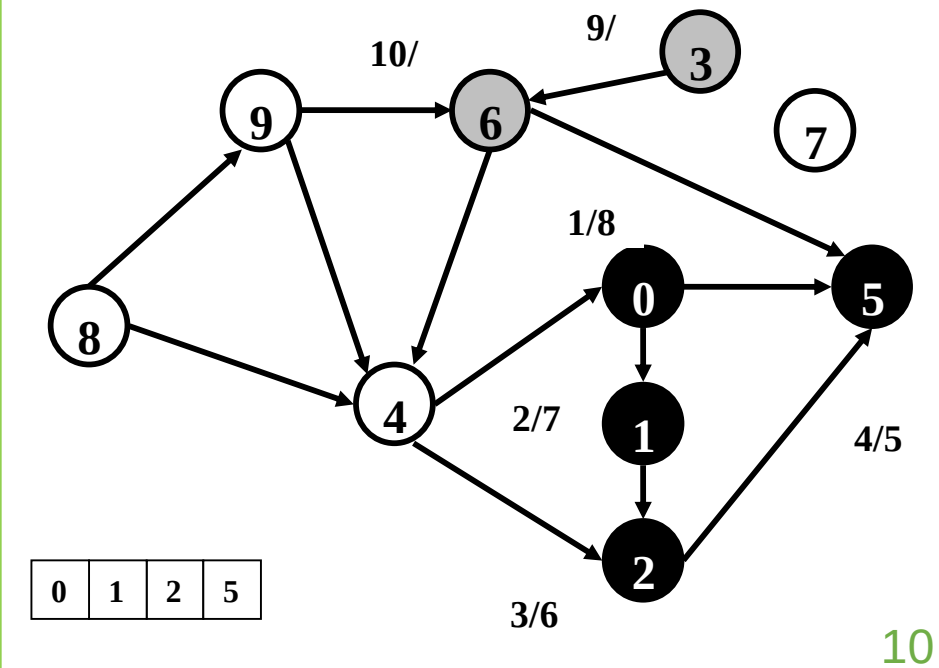
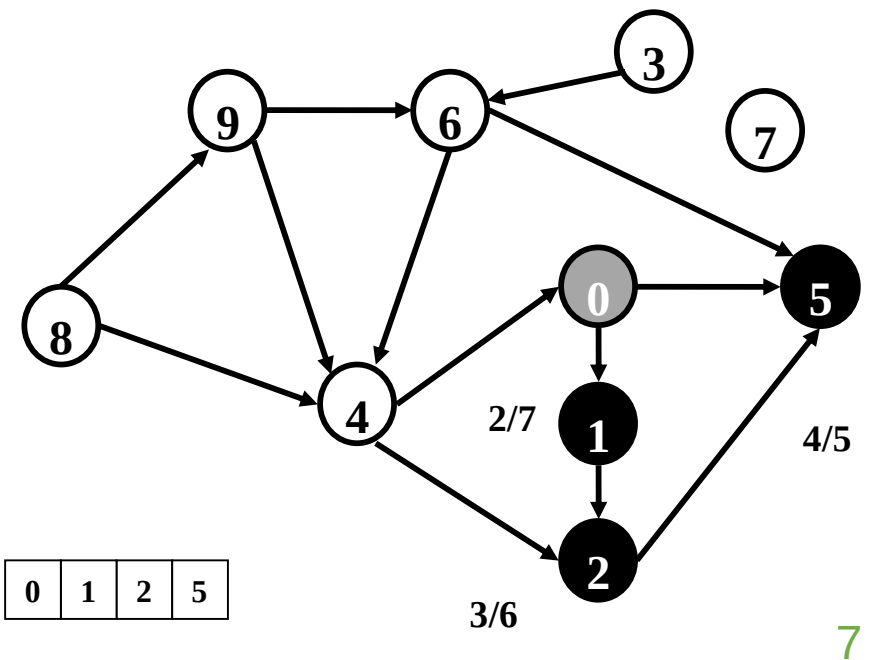
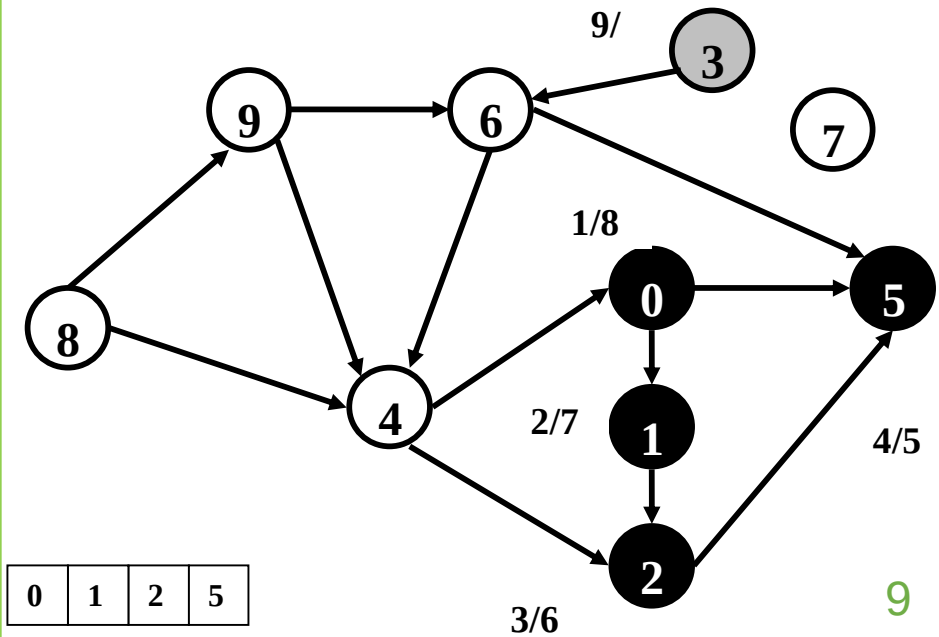
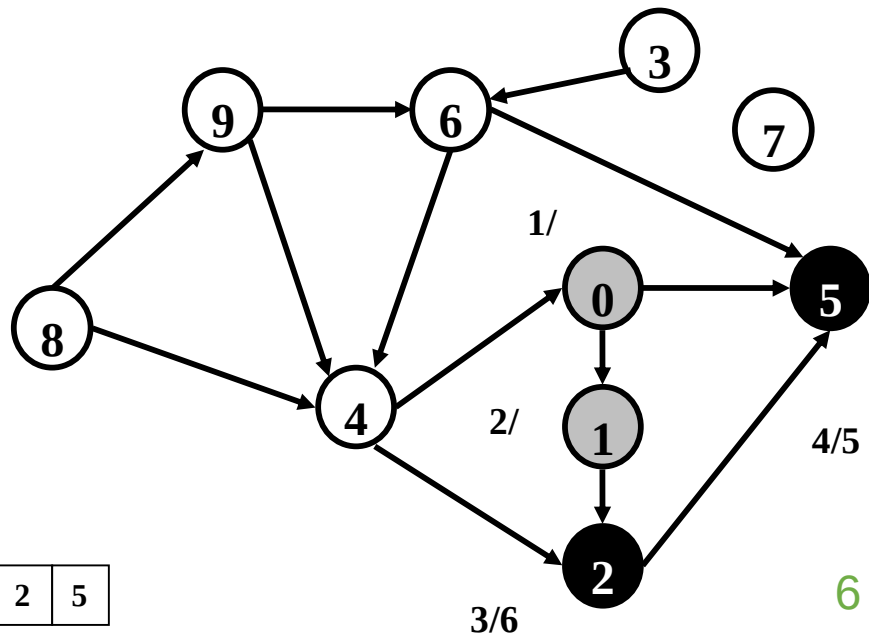
4

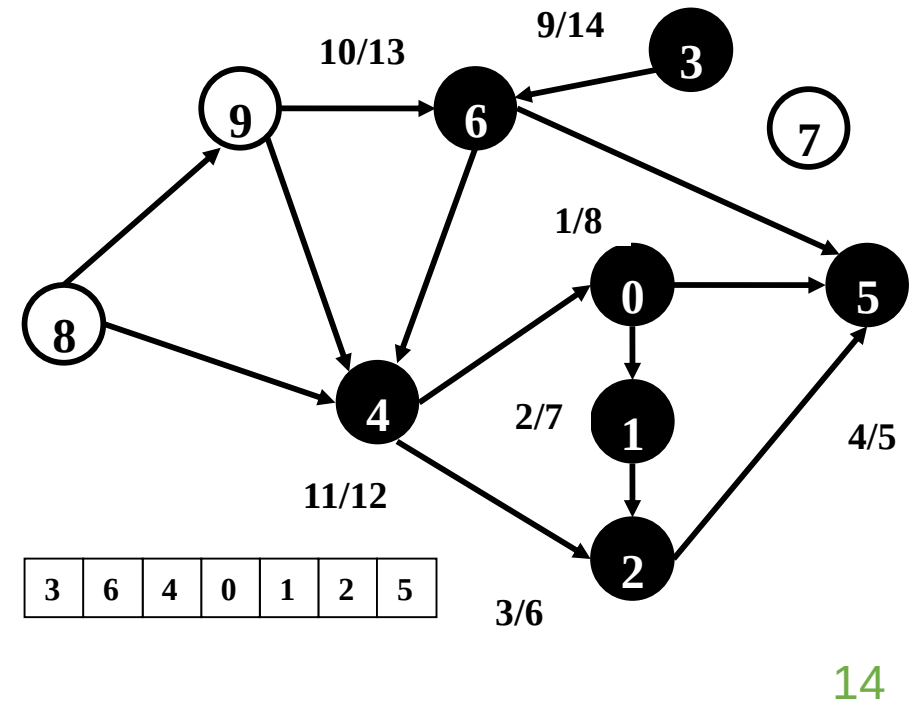
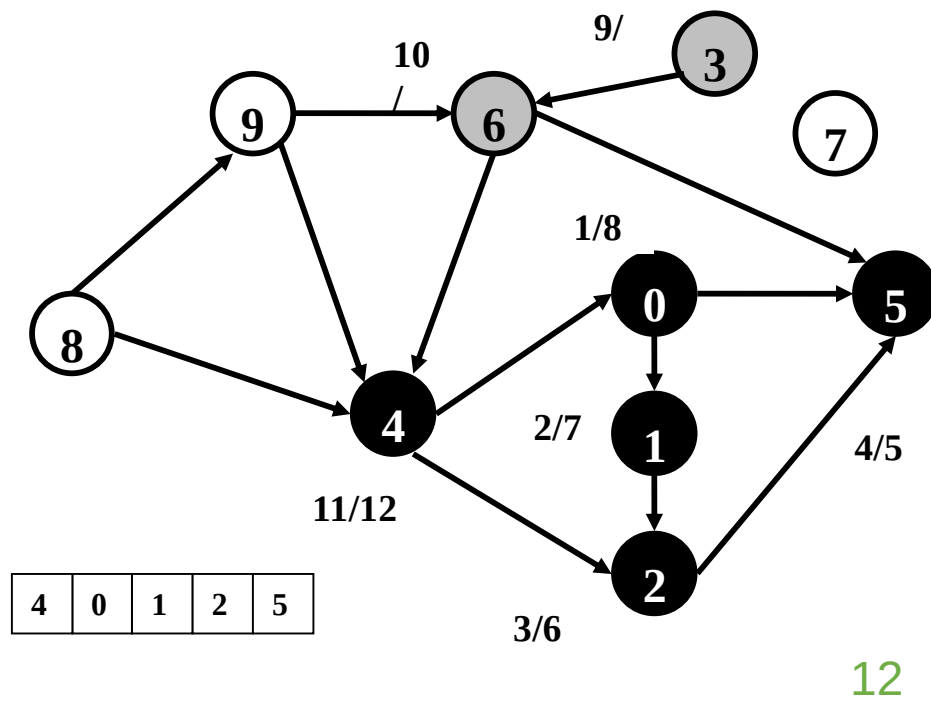
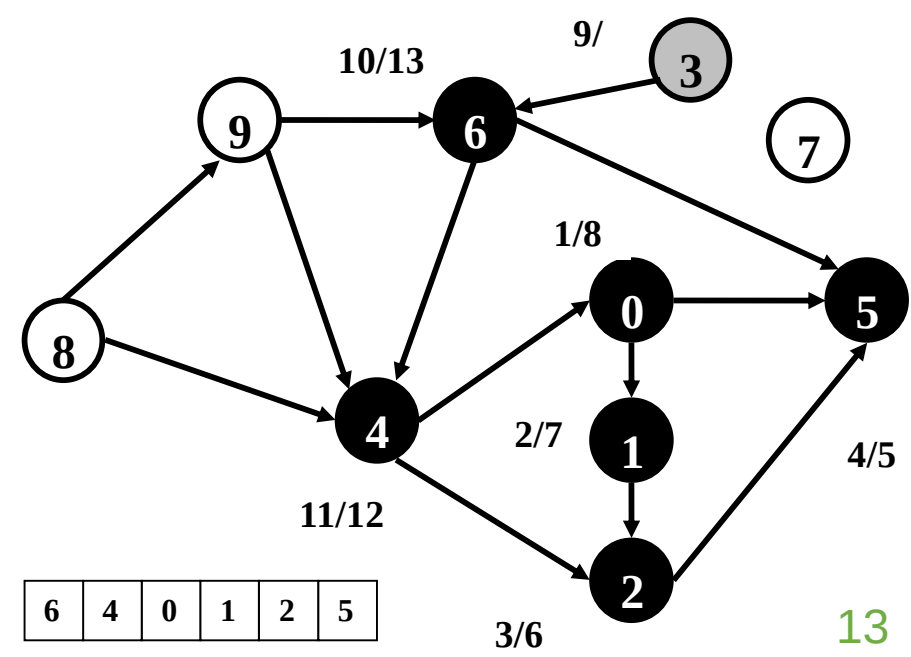
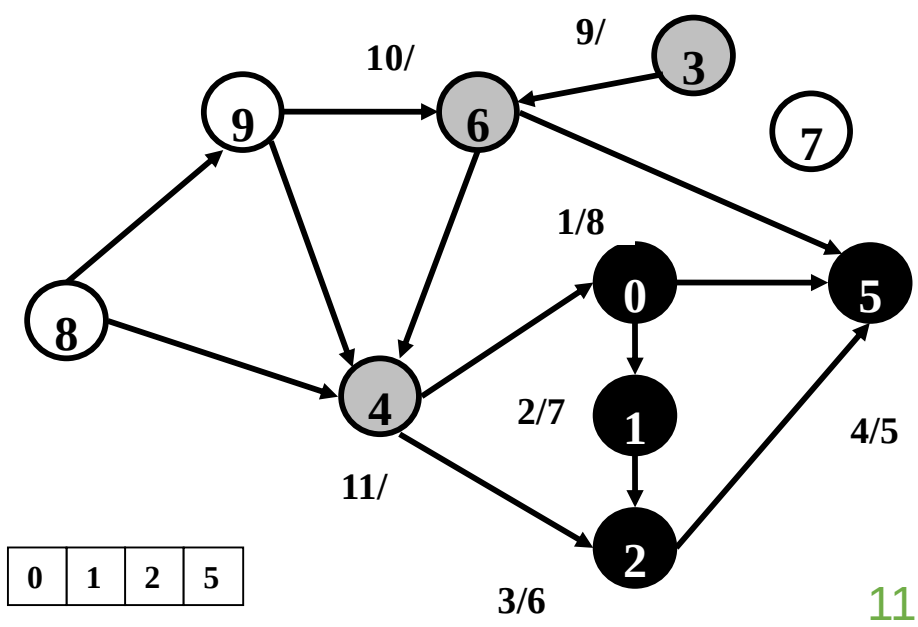


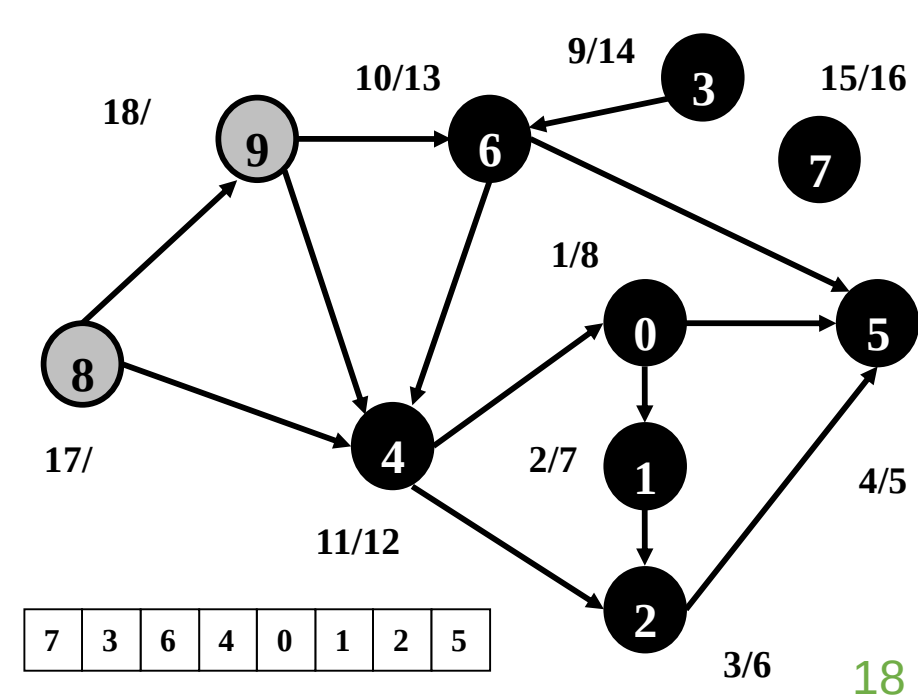
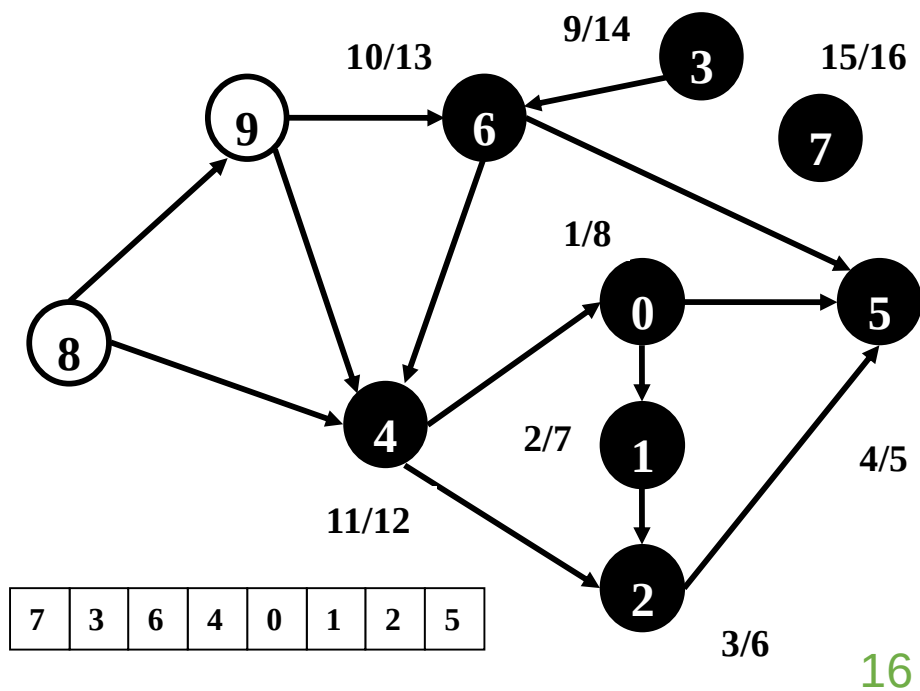
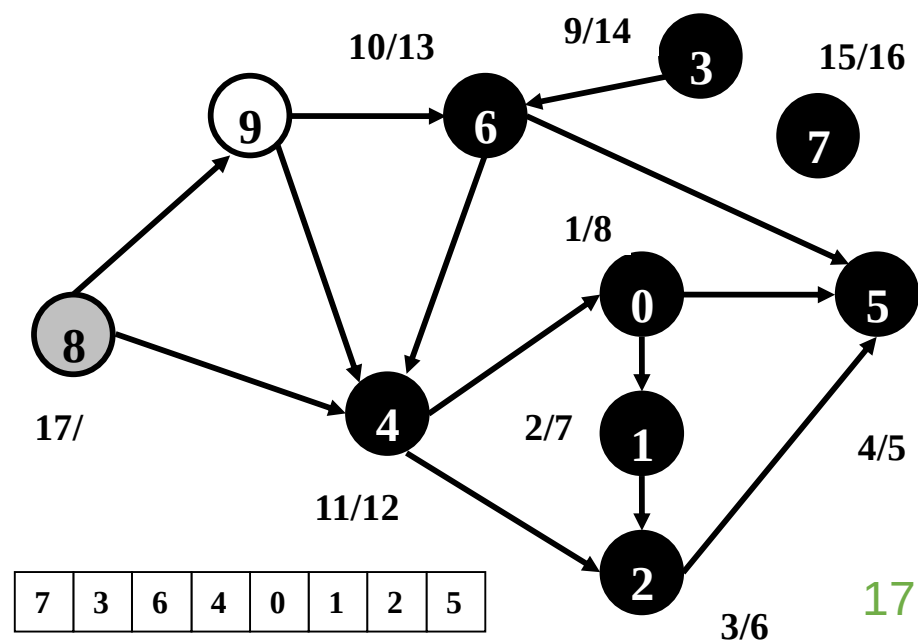
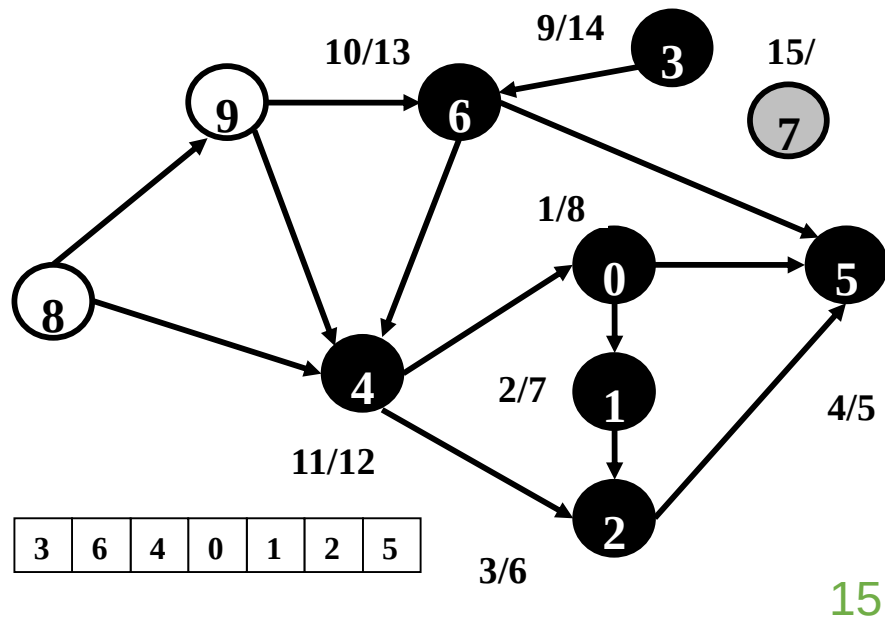
3

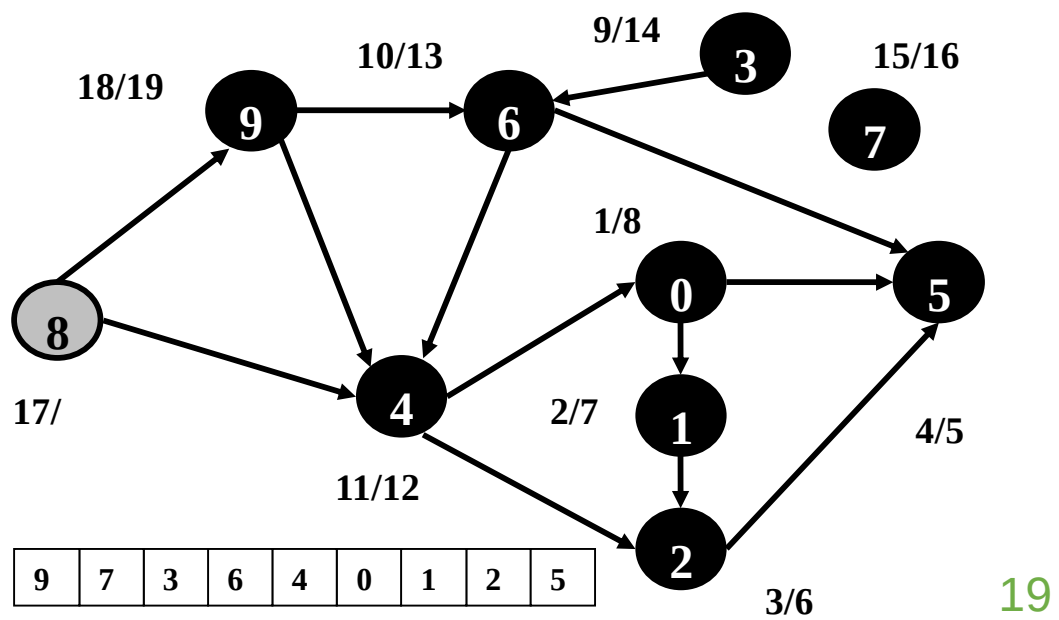


5

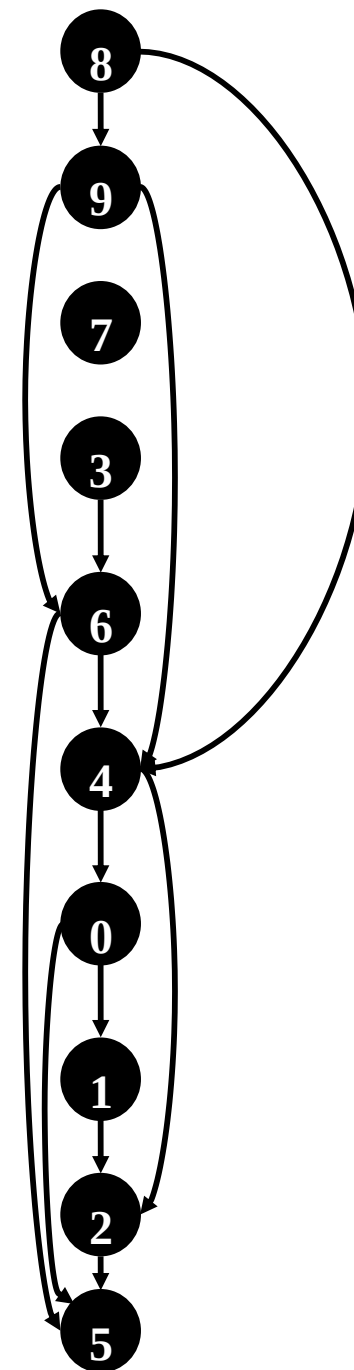
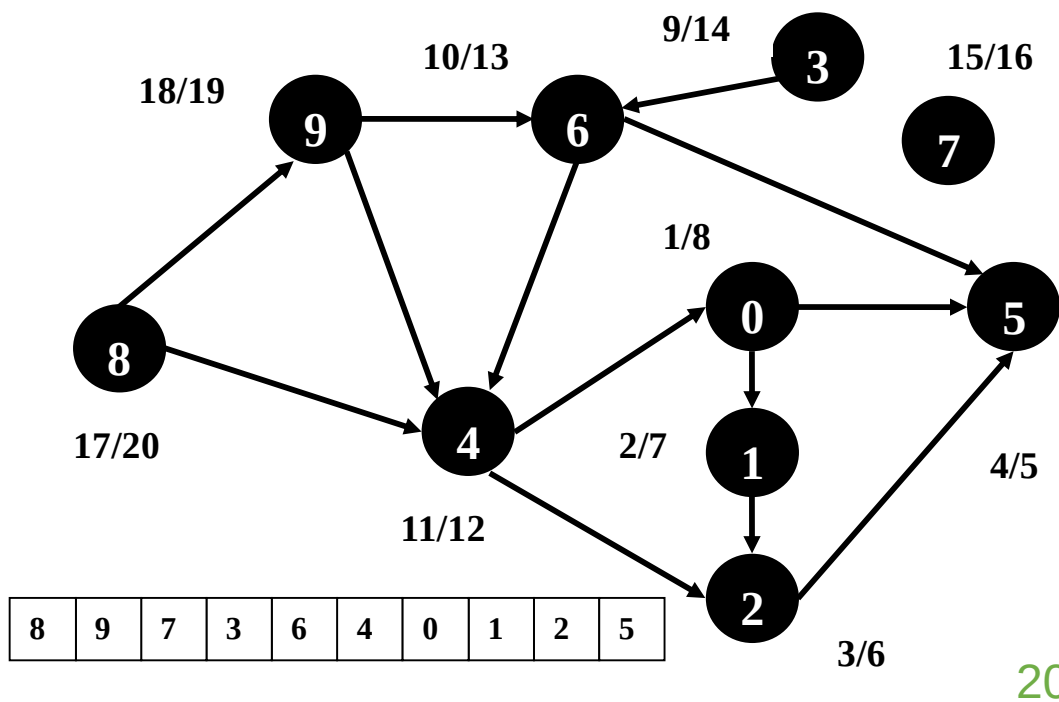








8
9
7
3
6
4
0
1
2
5



21